



# **ShopSense: E-Commerce Sales and Customer Behaviour Analytics Using Big Data and Machine Learning**

**LY Innovation-Based Major Project-B Report**

**Submitted in partial fulfillment of the requirements of the Degree of  
Bachelor of Technology in Computer Engineering**

by

**Kartik Verma**

**Kushal Soni**

**Dhir Thakar**

**Supervisor**

**<Guide Name>**



Department of Computer Engineering

K. J. Somaiya Institute of Technology

**An Autonomous Institute permanently affiliated to University of Mumbai**

**Ayurvihar, Sion, Mumbai -400022**

**2026-27**



# **ShopSense: E-Commerce Sales and Customer Behaviour Analytics Using Big Data and Machine Learning**

**LY Innovation-Based Major Project-B Report**

**Submitted in partial fulfillment of the requirements of the Degree of  
Bachelor of Technology in Computer Engineering**

by

**Kartik Verma (Roll No.)**

**Kushal Soni (Roll No.)**

**Dhir Thakar (Roll No.)**

**Supervisor**

**<Guide Name>**



**Department of Computer Engineering**

**K. J. Somaiya Institute of Technology**

**An Autonomous Institute permanently affiliated to University of Mumbai**

**Ayurvihar, Sion, Mumbai -400022**

**2026-27**



## CERTIFICATE



*This is to certify that the project entitled “**ShopSense: E-Commerce Sales and Customer Behaviour Analytics Using Big Data and Machine Learning**” is bonafide work of **Kartik Verma, Kushal Soni, Dhir Thakar** submitted to the University of Mumbai in partial fulfillment of the requirement in Major Project, for the award of the degree of “Bachelors of Technology” in “Computer Engineering”.*

---

<Guide Name>

Project Guide

Department of Computer Engineering

---

**Dr. Sarita Ambadekar,**  
**Head, Department of Computer**  
**Engineering**

---

**Dr. Vivek Sunnapwar**  
**Principal KJSIT**

Place: Sion, Mumbai-400022

Date:

## **PROJECT APPROVAL FOR L. Y.**

This project report entitled “**ShopSense: E-Commerce Sales and Customer Behaviour Analytics Using Big Data and Machine Learning**” by

**Kartik Verma (Roll No.)**

**Kushal Soni (Roll No.)**

**Dhir Thakar (Roll No.)**

is an approved Last Year Innovation-Based Major Project in Computer Engineering.

Examiners:

1. \_\_\_\_\_

Name and Signature

External Examiner

2. \_\_\_\_\_

Name and Signature

Internal Examiner

Place: Sion, Mumbai-400022

Date:

## DECLARATION

We declare that this written submission represents our ideas in our own words and where other's ideas or words have been included, we have adequately cited and referenced the sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

---

**Kartik Verma**

---

**Kushal Soni**

---

**Dhir Thakar**

Date:

## ACKNOWLEDGEMENT

Before presenting our LY Major Project work entitled “**ShopSense: E-Commerce Sales and Customer Behaviour Analytics Using Big Data and Machine Learning**”, we would like to convey our sincere thanks to the people who guided us throughout the course for this project work.

First, we would like to express our immense gratitude towards our Project Guide <**Guide Name**> for the constant encouragement, support, guidance, and mentoring at the ongoing stages of the project and report.

We would like to express our sincere thanks to our H.O.D. **Dr. Sarita Ambadekar**, for the encouragement, co-operation, and suggestions progressing stages of the report.

We would like to express our sincere thanks to our beloved Principal **Dr. Vivek Sunnapwar** for providing various facilities to carry out this project.

Finally, we would like to thank all the teaching and non-teaching staff of the college, and our friends, for their moral support rendered during the course of the reported work, and for their direct and indirect involvement in the completion of our report work, which made our endeavor fruitful.

Place: Sion, Mumbai-400022

Date:

## ABSTRACT

E-commerce platforms generate large volumes of heterogeneous data every day: product catalogues, customer profiles, order transactions and, above all, click-stream logs that record every page view, cart action and checkout. This project presents **ShopSense**, an end-to-end Big Data analytics platform that uses **Apache Spark**, **MongoDB** and **machine learning** to analyse e-commerce sales and customer behaviour.

A behaviour-driven synthetic data generator produces a realistic two-year dataset for an Indian multi-category marketplace (10,000 customers, 74,348 orders and 1.35 million click-stream events), with seasonality, festive sales, churn behaviour and injected data-quality problems. A PySpark pipeline organised as bronze, silver and gold layers ingests CSV and JSON-lines data, cleans and sessionises it and computes analytics tables that are written in parallel to MongoDB as query-ready documents. Six Spark MLlib models add intelligence: K-Means customer segmentation on RFM features (silhouette 0.375), churn prediction comparing Logistic Regression, Random Forest and Gradient-Boosted Trees with a leak-free time-based split (best ROC-AUC 0.955), implicit-feedback ALS product recommendations (HitRate@10 0.180, 1.8× a popularity baseline), FP-Growth market-basket rules, a hybrid trend-and-seasonality sales forecaster with prediction intervals, and model-based anomaly detection that recovered all planted anomalies without false alarms.

The results are served through a FastAPI REST interface and an interactive eleven-page dashboard offering sales, funnel, cohort, segmentation, churn, recommendation, market-basket and forecasting views, a Customer 360 profile, a live what-if churn simulator and exportable win-back campaign lists. The pipeline processes about 1.4 million raw records in roughly seven minutes on a laptop, while dashboard queries are answered in milliseconds.

**Keywords:** Big Data, Apache Spark, MongoDB, Machine Learning, Customer Segmentation, Churn Prediction, Recommender Systems, Market Basket Analysis, Sales Forecasting, E-Commerce Analytics.

# CONTENTS

| Chapter No. |     | TITLE                                | Page No.    |
|-------------|-----|--------------------------------------|-------------|
|             |     | <b>LIST OF FIGURES</b>               | <b>viii</b> |
|             |     | <b>LIST OF TABLES</b>                | <b>ix</b>   |
|             |     | <b>LIST OF ABBREVIATIONS</b>         | <b>x</b>    |
|             |     |                                      |             |
| <b>1</b>    |     | <b>INTRODUCTION</b>                  | <b>1</b>    |
|             | 1.1 | Introduction                         | 1           |
|             | 1.2 | Problem Definition                   | 2           |
|             | 1.3 | Aim and Objectives                   | 3           |
|             | 1.4 | Organization of the Report           | 4           |
|             |     |                                      |             |
| <b>2</b>    |     | <b>REVIEW OF LITERATURE</b>          | <b>5</b>    |
|             | 2.1 | Literature Survey                    | 5           |
|             | 2.2 | Research Gaps                        | 9           |
|             |     |                                      |             |
| <b>3</b>    |     | <b>REQUIREMENT SPECIFICATION</b>     | <b>10</b>   |
|             | 3.1 | Introduction                         | 10          |
|             | 3.2 | Hardware Requirements                | 10          |
|             | 3.3 | Software Requirements                | 10          |
|             | 3.4 | Feasibility Study                    | 11          |
|             | 3.5 | Cost Estimation                      | 12          |
|             | 3.6 | Functional Requirements              | 13          |
|             | 3.7 | Non-Functional Requirements          | 14          |
|             | 3.8 | Dataset Specification                | 14          |
|             |     |                                      |             |
| <b>4</b>    |     | <b>PROJECT ANALYSIS &amp; DESIGN</b> | <b>16</b>   |
|             | 4.1 | Introduction                         | 16          |
|             | 4.2 | UML Diagrams                         | 16          |
|             | 4.3 | System Architecture                  | 23          |
|             | 4.4 | Database Design                      | 24          |
|             | 4.5 | Deployment Design                    | 26          |
|             | 4.6 | User Interface Design                | 27          |
|             |     |                                      |             |
| <b>5</b>    |     | <b>METHODOLOGY</b>                   | <b>28</b>   |
|             | 5.1 | Introduction                         | 28          |
|             | 5.2 | System Block Diagram                 | 28          |

|          |      |                                        |           |
|----------|------|----------------------------------------|-----------|
|          | 5.3  | Data Generation Model                  | 28        |
|          | 5.4  | Data Ingestion and Cleaning            | 29        |
|          | 5.5  | Descriptive Analytics                  | 30        |
|          | 5.6  | Customer Segmentation using K-Means    | 31        |
|          | 5.7  | Churn Prediction                       | 32        |
|          | 5.8  | Product Recommendation using ALS       | 33        |
|          | 5.9  | Market-Basket Analysis using FP-Growth | 34        |
|          | 5.10 | Sales Forecasting                      | 34        |
|          | 5.11 | Anomaly Detection                      | 35        |
|          | 5.12 | Distributed Loading into MongoDB       | 35        |
|          |      |                                        |           |
| <b>6</b> |      | <b>IMPLEMENTATION DETAILS</b>          | <b>37</b> |
|          | 6.1  | Introduction                           | 37        |
|          | 6.2  | System Implementation                  | 37        |
|          | 6.3  | Project Structure                      | 46        |
|          | 6.4  | Key Implementation Details             | 46        |
|          | 6.5  | REST API                               | 47        |
|          | 6.6  | Testing                                | 48        |
|          |      |                                        |           |
| <b>7</b> |      | <b>RESULT ANALYSIS</b>                 | <b>49</b> |
|          | 7.1  | Introduction                           | 49        |
|          | 7.2  | Performance Measures                   | 49        |
|          | 7.3  | Screenshots                            | 58        |
|          | 7.4  | Comparative Analysis                   | 58        |
|          |      |                                        |           |
| <b>8</b> |      | <b>CONCLUSION &amp; FUTURE SCOPE</b>   | <b>60</b> |
|          | 8.1  | Conclusion                             | 60        |
|          | 8.2  | Future Scope                           | 60        |
|          |      |                                        |           |
|          |      | <b>REFERENCES</b>                      | <b>62</b> |
|          |      |                                        |           |
|          |      | <b>PUBLISHED PAPERS</b>                | <b>65</b> |
|          |      |                                        |           |
|          |      | <b>CERTIFICATES</b>                    | <b>66</b> |
|          |      |                                        |           |
|          |      | <b>PLAGIARISM REPORT</b>               | <b>67</b> |

## LIST OF FIGURES

| <b>Figure No.</b> | <b>Title</b>                                             | <b>Page No.</b> |
|-------------------|----------------------------------------------------------|-----------------|
| 3.1               | Project schedule (Gantt chart)                           | 12              |
| 4.1               | Use-case diagram of ShopSense                            | 17              |
| 4.2               | Sequence diagram: execution of the Spark pipeline        | 18              |
| 4.3               | Sequence diagram: churn dashboard and what-if simulator  | 19              |
| 4.4               | State diagrams                                           | 20              |
| 4.5               | Activity diagram: churn prediction workflow              | 21              |
| 4.6               | Class diagram of the ShopSense modules                   | 22              |
| 4.7               | Data flow diagram – level 0 (context diagram)            | 22              |
| 4.8               | Data flow diagram – level 1                              | 23              |
| 4.9               | System architecture of ShopSense                         | 24              |
| 4.10              | Logical data model of the main MongoDB collections       | 25              |
| 4.11              | Deployment diagram                                       | 27              |
| 5.1               | System block diagram (bronze – silver – gold data flow)  | 28              |
| 6.1               | Console output of a complete pipeline run                | 37              |
| 6.2               | MongoDB collections and an embedded order document       | 38              |
| 6.3               | Overview dashboard                                       | 39              |
| 6.4               | Sales analytics page with filters                        | 39              |
| 6.5               | Product performance page                                 | 40              |
| 6.6               | Customer behaviour page                                  | 41              |
| 6.7               | Customer segmentation page                               | 41              |
| 6.8               | Churn prediction page                                    | 42              |
| 6.9               | What-if churn simulator with prediction drivers          | 42              |
| 6.10              | Personalised recommendations page                        | 43              |
| 6.11              | Market-basket analysis page                              | 43              |
| 6.12              | Forecasting and anomaly-detection page                   | 44              |
| 6.13              | Customer 360 page of an at-risk high-value customer      | 45              |
| 6.14              | Data pipeline and model monitoring page                  | 45              |
| 6.15              | Interactive REST API documentation (Swagger UI)          | 46              |
| 7.1               | Order-volume uplift versus revenue uplift of sale events | 52              |
| 7.2               | Conversion funnel for all sessions                       | 52              |
| 7.3               | ROC curves of the three churn models                     | 54              |
| 7.4               | Feature importance of the Random Forest churn model      | 55              |
| 7.5               | Revenue forecast with 80% and 95% prediction intervals   | 57              |
| 7.6               | Anomaly timeline: daily revenue with detected anomalies  | 58              |

## LIST OF TABLES

| <b>Table No.</b> | <b>Title</b>                                              | <b>Page No.</b> |
|------------------|-----------------------------------------------------------|-----------------|
| 2.1              | Summary of literature survey                              | 8               |
| 3.1              | Hardware requirements                                     | 10              |
| 3.2              | Software requirements                                     | 10              |
| 3.3              | Cost estimation                                           | 12              |
| 3.4              | Functional requirements                                   | 13              |
| 3.5              | Raw dataset specification (medium scale)                  | 14              |
| 4.1              | MongoDB collections produced by the pipeline              | 25              |
| 5.1              | Rules for naming clusters from their centroids            | 31              |
| 5.2              | Churn prediction features                                 | 32              |
| 6.1              | Project modules                                           | 46              |
| 6.2              | Representative test cases                                 | 48              |
| 7.1              | Pipeline stage timings (medium dataset)                   | 49              |
| 7.2              | Scalability across dataset sizes                          | 50              |
| 7.3              | Data-quality report                                       | 50              |
| 7.4              | Key performance indicators (July 2024 – June 2026)        | 51              |
| 7.5              | Category performance                                      | 51              |
| 7.6              | Silhouette coefficient for different numbers of clusters  | 53              |
| 7.7              | Customer segments discovered by K-Means                   | 53              |
| 7.8              | Churn model comparison on the test set                    | 53              |
| 7.9              | Recommendation accuracy (Top-10, leave-last-purchase-out) | 55              |
| 7.10             | Strongest sub-category association rules                  | 56              |
| 7.11             | Forecast accuracy on the 60-day hold-out window           | 56              |
| 7.12             | Detected anomalous days versus planted ground truth       | 57              |
| 7.13             | Comparison with existing approaches                       | 58              |

## LIST OF ABBREVIATIONS

| Sr. No | Abbreviation | Description                           |
|--------|--------------|---------------------------------------|
| 1      | ALS          | Alternating Least Squares             |
| 2      | API          | Application Programming Interface     |
| 3      | AOV          | Average Order Value                   |
| 4      | AUC          | Area Under the (ROC) Curve            |
| 5      | BSON         | Binary JSON                           |
| 6      | CLV          | Customer Lifetime Value               |
| 7      | COD          | Cash on Delivery                      |
| 8      | CSV          | Comma-Separated Values                |
| 9      | CV           | Cross-Validation                      |
| 10     | DFD          | Data Flow Diagram                     |
| 11     | DQ           | Data Quality                          |
| 12     | ETL          | Extract, Transform, Load              |
| 13     | FP-Growth    | Frequent Pattern Growth               |
| 14     | GBT          | Gradient-Boosted Trees                |
| 15     | HDFS         | Hadoop Distributed File System        |
| 16     | IQR          | Inter-Quartile Range                  |
| 17     | JDK          | Java Development Kit                  |
| 18     | JSON         | JavaScript Object Notation            |
| 19     | JVM          | Java Virtual Machine                  |
| 20     | KPI          | Key Performance Indicator             |
| 21     | LR           | Logistic Regression                   |
| 22     | MAD          | Median Absolute Deviation             |
| 23     | MAE          | Mean Absolute Error                   |
| 24     | MAPE         | Mean Absolute Percentage Error        |
| 25     | ML           | Machine Learning                      |
| 26     | MLlib        | Spark Machine Learning Library        |
| 27     | NDCG         | Normalised Discounted Cumulative Gain |
| 28     | NoSQL        | Not only SQL                          |
| 29     | PCA          | Principal Component Analysis          |
| 30     | PR-AUC       | Area Under the Precision-Recall Curve |
| 31     | RDD          | Resilient Distributed Dataset         |
| 32     | REST         | Representational State Transfer       |
| 33     | RF           | Random Forest                         |
| 34     | RFM          | Recency, Frequency, Monetary          |
| 35     | RMSE         | Root Mean Squared Error               |
| 36     | ROC          | Receiver Operating Characteristic     |

|    |     |                            |
|----|-----|----------------------------|
| 37 | SQL | Structured Query Language  |
| 38 | UI  | User Interface             |
| 39 | UML | Unified Modeling Language  |
| 40 | UPI | Unified Payments Interface |

# CHAPTER 1

## INTRODUCTION

### 1.1 Introduction

Electronic commerce has become one of the fastest-growing sectors of the Indian economy. Driven by affordable smartphones, the Unified Payments Interface (UPI) and cheap mobile data, millions of first-time buyers from tier-2 and tier-3 cities now shop online for electronics, fashion, groceries and daily essentials. Every interaction on an e-commerce platform leaves a digital trace: a product page view, an item added to the cart, a coupon applied, an order placed, cancelled or returned. A single mid-sized marketplace can generate millions of such events every month, in different formats (relational order tables, semi-structured JSON click-stream logs and product catalogues) and at a velocity that traditional spreadsheet or single-server relational tools cannot handle comfortably. These are exactly the *volume*, *variety* and *velocity* characteristics that define **Big Data**.

The value of this data lies in the questions it can answer. Which categories and cities drive revenue? At which step of the purchase funnel do shoppers abandon their carts? Who are the most valuable customers, and which of them are about to stop buying? Which products should be recommended to each customer, and which items are usually bought together? How much revenue should the business expect next month, and was yesterday's drop in orders normal or a sign of a technical failure? Answering these questions requires two capabilities working together: **distributed data processing** to clean and aggregate large, messy datasets, and **machine learning** to discover patterns and make predictions. This project, *ShopSense*, combines **Apache Spark** (for distributed processing and machine learning through Spark MLlib), **MongoDB** (a document-oriented NoSQL database used as the serving layer) and a modern web dashboard to build a complete, end-to-end *E-Commerce Sales and Customer Behaviour Analytics* platform.

The domain of this project therefore lies at the intersection of Big Data engineering, business intelligence and applied machine learning. The system follows the same architectural ideas used by large online retailers (a layered data lake, batch model training and a low-latency serving store) but is designed to run on a student laptop with

only open-source software, which makes it both practical to demonstrate and easy to extend.

## 1.2 Problem Definition

Small and medium online retailers collect large amounts of transactional and behavioural data but lack an affordable, integrated way to convert it into decisions. The specific problems addressed by this project are:

- **Scale and variety of data:** order tables, product catalogues, customer profiles and more than a million semi-structured click-stream events cannot be processed interactively with spreadsheets or a single relational database.
- **Poor data quality:** raw exports contain duplicate rows, inconsistent spellings (for example “upi”, “ UPI ” and “UPI”), missing values and invalid quantities that silently corrupt reports.
- **Fragmented analytics:** sales reports, funnel analysis, customer segmentation and forecasting are usually produced by separate tools, so managers never see a single, consistent picture.
- **Reactive customer management:** businesses notice that a valuable customer has left only after revenue has already been lost; there is no early-warning signal or explanation of *why* a customer is at risk.
- **Generic merchandising:** every visitor sees the same best-sellers, and bundle offers are designed by intuition instead of from actual co-purchase patterns.
- **Planning without foresight:** revenue targets and inventory are planned without a forecast that accounts for weekends, salary days and festive sale events, and unusual days (outages or bulk orders) are not detected automatically.

The problem is therefore to **design and implement a scalable Big Data analytics system that ingests heterogeneous e-commerce data, cleans it, derives business analytics and machine-learning insights with Apache Spark, stores the results in MongoDB and presents them through an interactive, decision-oriented dashboard.**

### 1.3 Aim and Objectives

**Aim:** To develop *ShopSense*, an end-to-end platform that uses Apache Spark, MongoDB and machine learning to analyse e-commerce sales and customer behaviour and to deliver predictive, actionable insights through an interactive web dashboard.

**Objectives:**

1. To generate a realistic, reproducible e-commerce dataset (customers, products, orders, order items and click-stream events) that contains seasonality, customer life-cycles and data-quality issues.
2. To build a layered (bronze–silver–gold) Apache Spark pipeline that ingests CSV and JSON-lines data, validates and cleans it, and produces a measurable data-quality report.
3. To compute sales and behavioural analytics (KPIs, trends, geography, conversion funnels, sessions and cohort retention) using Spark SQL and window functions.
4. To design a MongoDB document model and write the analytical results to it in a distributed manner, so that dashboard queries are answered in milliseconds.
5. To segment customers with RFM features and K-Means clustering, and to translate each segment into a marketing action.
6. To predict customer churn with a leak-free, time-based methodology, compare Logistic Regression, Random Forest and Gradient-Boosted Trees, and explain individual predictions.
7. To recommend products with implicit-feedback ALS collaborative filtering and to mine frequently-bought-together rules with FP-Growth.
8. To forecast daily revenue with prediction intervals and to detect anomalous days and suspicious orders automatically.
9. To expose all results through a documented REST API and an eleven-page interactive dashboard with a what-if churn simulator and campaign export.

**Scope:** ShopSense covers the complete analytics life-cycle for an online marketplace: data generation or ingestion, distributed cleaning and transformation, descriptive analytics, predictive modelling, storage and visualisation. It operates as a *batch* system that is re-run periodically (for example nightly). Real-time streaming ingestion, payment or inventory management and deployment on a multi-node production cluster

are outside the present scope, although the architecture has been designed so that these can be added (Chapter 8). Because the dataset is synthetic, no personal data of real customers is processed.

## 1.4 Organization of the Report

The rest of this report is organised as follows. **Chapter 2** reviews the literature on Big Data processing frameworks, NoSQL storage, customer segmentation, churn prediction, recommender systems, association-rule mining, forecasting and anomaly detection, and identifies the research gaps addressed by this work. **Chapter 3** specifies the hardware, software, functional and non-functional requirements and presents the feasibility study and cost estimation. **Chapter 4** describes the analysis and design of the system through UML diagrams, data-flow diagrams and the database design. **Chapter 5** explains the methodology, including the system block diagram, algorithms and mathematical formulations of every model. **Chapter 6** presents the implementation details with screenshots of the application. **Chapter 7** analyses the results using quantitative performance measures and compares them with existing approaches. **Chapter 8** concludes the report and outlines the future scope.

## CHAPTER 2

### REVIEW OF LITERATURE

#### 2.1 Literature Survey

**Survey strategy.** The literature survey was carried out in three stages. First, foundational papers on distributed data processing and NoSQL storage were identified to justify the technology stack. Second, for each analytical task in the project (segmentation, churn prediction, recommendation, association-rule mining, forecasting and anomaly detection) the seminal algorithm papers and widely cited applied studies were collected from IEEE Xplore, the ACM Digital Library, Elsevier ScienceDirect, Springer and the official documentation of Apache Spark and MongoDB, using keywords such as “Spark MLlib”, “RFM segmentation”, “customer churn prediction”, “implicit feedback collaborative filtering”, “FP-Growth” and “robust anomaly detection”. Third, each paper was studied for its method, data set, evaluation metric and limitations, and the findings were consolidated into the summary table (Table 2.1) and the research gaps (Section 2.2).

##### 2.1.1 Big Data processing frameworks

Dean and Ghemawat [3] introduced **MapReduce**, a programming model in which a computation is expressed as a *map* and a *reduce* function and executed in parallel over a cluster, with automatic fault tolerance. MapReduce made large-scale batch processing accessible but writes intermediate results to disk between stages, which makes iterative algorithms such as machine learning slow. Zaharia et al. [1] addressed this limitation with **Resilient Distributed Datasets (RDDs)**, an in-memory abstraction that records the lineage of transformations so that lost partitions can be recomputed; iterative workloads ran up to twenty times faster than on Hadoop MapReduce. The same group later described **Apache Spark** as a unified engine [2] that supports SQL, streaming, graph processing and machine learning on the same core, and Armbrust et al. [4] presented **Spark SQL**, whose DataFrame API and Catalyst optimiser combine relational queries with procedural code. Armbrust et al. [25] further proposed the *lakehouse* architecture, in which raw data is refined in successive layers on open file formats, an idea that ShopSense applies through its bronze, silver and gold layers.

### 2.1.2 Machine learning at scale and NoSQL storage

Meng et al. [5] described **MLlib**, Spark’s distributed machine-learning library, which provides scalable implementations of classification, regression, clustering, collaborative filtering and frequent-pattern mining together with a *Pipeline* API for feature transformation, model tuning and evaluation. MLlib allows the same code to run on a laptop and on a cluster, which is essential for this project. For storage, Bradshaw, Brazil and Chodorow [6] describe **MongoDB**, a document database that stores flexible JSON-like (BSON) documents, supports secondary indexes and provides a powerful aggregation pipeline [28]. Document databases are well suited to serving pre-aggregated analytics because each dashboard widget can be answered with a single indexed look-up, and related data (for example an order and its line items) can be embedded in one document.

### 2.1.3 Customer segmentation

Chen, Sain and Guo [7] segmented the customers of a UK online retailer using the **RFM** model (Recency, Frequency, Monetary value) followed by k-means clustering and decision-tree induction, showing that RFM clusters correspond to meaningful business groups. Fader, Hardie and Lee [8] linked RFM variables to **customer lifetime value (CLV)** and showed that recency and frequency jointly determine a customer’s future value. For the clustering algorithm, Bahmani et al. [9] proposed **k-means||** (scalable k-means++), a parallel initialisation that is used by Spark MLlib, and Rousseeuw [10] introduced the **silhouette coefficient**, the standard internal measure for choosing the number of clusters. A common limitation of applied segmentation studies is that clusters are labelled manually and are not linked to actions or other models.

### 2.1.4 Churn prediction

Verbeke et al. [11] benchmarked a large number of classifiers for churn prediction in the telecommunication sector and argued that models should be judged not only by accuracy but by their profitability and comprehensibility. Breiman’s **Random Forest** [12], an ensemble of decorrelated decision trees trained on bootstrap samples, and Friedman’s **gradient boosting** [13], which fits trees sequentially to the negative gradient of a loss function, are the strongest general-purpose learners for such tabular problems, while logistic regression remains the most interpretable baseline. Fawcett [14] provides the reference treatment of **ROC analysis**, the preferred evaluation

method when the classes are imbalanced or when the decision threshold is not fixed. Many published e-commerce churn studies, however, rely on randomly split snapshots in which features and labels overlap in time, which inflates the reported accuracy.

### 2.1.5 Recommender systems

Koren, Bell and Volinsky [15] popularised **matrix factorisation** for recommender systems during the Netflix Prize, representing users and items as latent-factor vectors whose inner product predicts preference. Most e-commerce platforms, however, do not have explicit star ratings; they have *implicit* signals such as views, carts and purchases. Hu, Koren and Volinsky [16] adapted matrix factorisation to **implicit feedback** by modelling a binary preference with a confidence weight and solving it efficiently with **Alternating Least Squares (ALS)**, the algorithm implemented in Spark MLlib. Their evaluation also showed that popularity-based recommendation is a surprisingly strong baseline that any personalised model must beat.

### 2.1.6 Market-basket analysis

Agrawal and Srikant [17] introduced the **Apriori** algorithm for mining association rules from transaction databases using the support and confidence measures; Apriori generates and tests candidate itemsets level by level, which becomes expensive for large databases. Han, Pei and Yin [18] proposed **FP-Growth**, which compresses the database into a frequent-pattern tree and mines it without candidate generation, and Li et al. [19] parallelised it as **PFP**, the version provided by Spark MLlib.

### 2.1.7 Sales forecasting and anomaly detection

Hyndman and Athanasopoulos [21] give a comprehensive treatment of forecasting methods, from simple benchmarks (naïve, seasonal naïve and moving averages) to regression and ARIMA models, and stress that every model must be compared against such benchmarks on a hold-out period. Taylor and Letham [20] described **Prophet**, an additive model with trend, seasonality and *holiday* components designed for business time series, demonstrating the importance of explicitly modelling events such as promotions. Duan [24] showed that predictions made on a logarithmic scale are biased when transformed back and proposed the **smearing estimator** to correct them. For anomaly detection, Chandola, Banerjee and Kumar [22] surveyed the field and classified anomalies into point, contextual and collective anomalies, while Leys et al. [23] recommended the **median absolute deviation (MAD)** instead of the standard

deviation, because the mean and standard deviation are themselves distorted by the outliers one is trying to find.

Table 2.1 summarises the reviewed literature and its relevance to the present work.

**Table 2.1:** Summary of literature survey

| Sr. | Authors (Year)                     | Technique / Contribution                                                | Relevance and limitation                                             |
|-----|------------------------------------|-------------------------------------------------------------------------|----------------------------------------------------------------------|
| 1   | Dean & Ghemawat (2004) [3]         | MapReduce programming model for cluster computing                       | Foundation of Big Data processing; disk-based, slow for iterative ML |
| 2   | Zaharia et al. (2012) [1]          | Resilient Distributed Datasets, in-memory lineage-based fault tolerance | Basis of Spark; up to 20× faster iterative jobs                      |
| 3   | Zaharia et al. (2016) [2]          | Apache Spark as a unified engine (SQL, ML, streaming)                   | Chosen processing engine for ShopSense                               |
| 4   | Armbrust et al. (2015) [4]         | Spark SQL, DataFrames and Catalyst optimiser                            | Used for all ETL and aggregations                                    |
| 5   | Meng et al. (2016) [5]             | MLlib: distributed ML library and Pipeline API                          | All six models built with MLlib                                      |
| 6   | Bradshaw et al. (2019) [6]         | MongoDB document model, indexing, aggregation                           | Serving layer; embedded documents                                    |
| 7   | Chen, Sain & Guo (2012) [7]        | RFM + k-means segmentation of online retail customers                   | Validates RFM clustering; manual labelling of clusters               |
| 8   | Fader, Hardie & Lee (2005) [8]     | Relationship between RFM and customer lifetime value                    | Motivates CLV estimate                                               |
| 9   | Bahmani et al. (2012) [9]          | Scalable k-means   initialisation                                       | Initialisation used by Spark K-Means                                 |
| 10  | Rousseeuw (1987) [10]              | Silhouette coefficient for cluster validation                           | Used to select k                                                     |
| 11  | Verbeke et al. (2012) [11]         | Benchmark of churn classifiers; profit-driven evaluation                | Stresses comprehensibility; telecom domain                           |
| 12  | Breiman (2001) [12]                | Random Forest ensemble                                                  | Best churn model in this project                                     |
| 13  | Friedman (2001) [13]               | Gradient boosting machines                                              | Compared churn model                                                 |
| 14  | Fawcett (2006) [14]                | ROC analysis for classifier evaluation                                  | Evaluation methodology                                               |
| 15  | Koren, Bell & Volinsky (2009) [15] | Matrix factorisation for recommendation                                 | Latent-factor foundation; explicit ratings                           |
| 16  | Hu, Koren & Volinsky (2008) [16]   | ALS for implicit feedback with confidence weights                       | Recommendation algorithm used                                        |
| 17  | Agrawal & Srikant (1994) [17]      | Apriori association-rule mining                                         | Support/confidence; candidate generation is costly                   |
| 18  | Han, Pei & Yin (2000) [18]         | FP-Growth without candidate generation                                  | Market-basket algorithm used                                         |
| 19  | Li et al. (2008) [19]              | Parallel FP-Growth (PFP)                                                | Spark implementation of FP-Growth                                    |
| 20  | Taylor & Letham (2018) [20]        | Prophet: trend + seasonality + holidays                                 | Inspires calendar-aware hybrid forecaster                            |

| Sr. | Authors (Year)                       | Technique / Contribution                          | Relevance and limitation          |
|-----|--------------------------------------|---------------------------------------------------|-----------------------------------|
| 21  | Hyndman & Athanasopoulos (2021) [21] | Forecasting principles, benchmarks and evaluation | Baselines and hold-out evaluation |
| 22  | Duan (1983) [24]                     | Smearing retransformation estimator               | Bias correction of log forecasts  |
| 23  | Chandola et al. (2009) [22]          | Survey of anomaly detection                       | Contextual anomaly framing        |
| 24  | Leys et al. (2013) [23]              | Median absolute deviation for outliers            | Robust z-score used               |
| 25  | Armbrust et al. (2021) [25]          | Lakehouse architecture with layered refinement    | Bronze/silver/gold design         |

## 2.2 Research Gaps

The review reveals the following gaps, each of which is addressed by ShopSense:

1. **Isolated solutions.** Most studies address one task (segmentation *or* churn *or* recommendation) on a static dataset. ShopSense feeds all models from one cleaned silver layer and combines their outputs in a Customer 360 profile.
2. **Temporal leakage in churn studies.** Many churn models are trained on randomly split snapshots in which the features already contain information from the label period. ShopSense uses a strict observation window / label window split and scores customers only with past data.
3. **Explicit-rating assumption.** Classical recommender evaluations rely on star ratings that e-commerce platforms rarely have. ShopSense uses implicit signals (views, carts, wish-lists, purchases) and evaluates with a leave-last-purchase-out protocol against a popularity baseline.
4. **Neglect of data quality.** Academic studies usually start from pre-cleaned datasets, while real exports are messy. ShopSense measures and reports every cleaning rule applied.
5. **Promotion-unaware forecasting and naïve anomaly detection.** Simple forecasts ignore planned sale events, and threshold-based anomaly detectors raise false alarms on every festival. ShopSense uses the marketing calendar as a feature, flags only deviations from a model expectation and turns every model output into an action (playbooks, explained scores, what-if simulation, campaign lists).

## CHAPTER 3

### REQUIREMENT SPECIFICATION

#### 3.1 Introduction

This chapter specifies the resources and requirements needed to build and run ShopSense. It lists the hardware and software requirements, analyses the technical, economic, operational and schedule feasibility of the project, estimates its cost with the COCOMO model and documents the functional and non-functional requirements as well as the specification of the input dataset.

#### 3.2 Hardware Requirements

Apache Spark runs in *local mode* on a single machine and uses all available CPU cores as parallel executors. The minimum and recommended configurations are given in Table 3.1; the results in this report were produced on a 4-core machine with 16 GB RAM.

**Table 3.1:** Hardware requirements

| Component | Minimum                         | Recommended                                       |
|-----------|---------------------------------|---------------------------------------------------|
| Processor | Dual-core x86-64, 2.0 GHz       | Quad-core (Intel Core i5 / AMD Ryzen 5) or better |
| RAM       | 4 GB (small dataset)            | 8–16 GB (medium / large dataset)                  |
| Storage   | 2 GB free disk space            | SSD with 5 GB free space                          |
| Display   | 1366 × 768                      | 1920 × 1080 (full dashboard layout)               |
| Network   | Not required after installation | Internet for installing packages                  |

#### 3.3 Software Requirements

**Table 3.2:** Software requirements

| Software             | Version                             | Purpose                            |
|----------------------|-------------------------------------|------------------------------------|
| Operating system     | Windows 10/11, macOS, Ubuntu 22.04+ | Host platform                      |
| Python               | 3.11 (3.9–3.11 supported)           | Pipeline, machine learning and API |
| Java Development Kit | OpenJDK 17 (11 also supported)      | Runtime for Apache Spark (JVM)     |

| Software                 | Version                               | Purpose                                        |
|--------------------------|---------------------------------------|------------------------------------------------|
| Apache Spark / PySpark   | 3.5.3                                 | Distributed ETL, Spark SQL and MLlib           |
| MongoDB Community Server | 6.0 / 7.0 / 8.0                       | Document database (serving layer)              |
| PyMongo                  | 4.6+                                  | MongoDB driver for Python                      |
| FastAPI + Uvicorn        | 0.110+ / 0.27+                        | REST API and web server                        |
| NumPy, python-dotenv     | 1.26+ / 1.0+                          | Numerical utilities, configuration             |
| Apache ECharts           | 5.6 (bundled)                         | Interactive charts in the dashboard            |
| Web browser              | Chrome, Edge, Firefox (latest)        | Dashboard client                               |
| Development tools        | VS Code, Git, pytest, MongoDB Compass | Coding, versioning, testing, database browsing |

### 3.4 Feasibility Study

#### 3.4.1 Technical feasibility

All components are mature, well-documented open-source technologies. PySpark bundles a complete Spark distribution, so no cluster installation is necessary; the same code can later run on a multi-node cluster by changing a single configuration value. MongoDB installs as a service on every major operating system. A prototype confirmed that the complete pipeline processes the medium dataset of about 1.4 million records in approximately seven minutes on a quad-core laptop. The project is therefore technically feasible within the skills of a final-year computer engineering team.

#### 3.4.2 Economic feasibility

The project uses only free and open-source software and runs on the students' existing laptops. No paid cloud service is required; if hosting is desired, MongoDB Atlas offers a free tier. The economic cost is limited to development effort (Section 3.5), which makes the project economically feasible, and the same platform would give a real retailer a low-cost alternative to commercial analytics suites.

#### 3.4.3 Operational feasibility

The dashboard is designed for non-technical business users: every page answers a business question, uses Indian currency formatting (lakh and crore), offers tooltips and filters, and provides downloadable campaign lists. The pipeline is operated with a single

command and records its own run log, so it can be scheduled nightly. Operational feasibility is therefore high.

### 3.4.4 Schedule feasibility

The work was divided into planning, data engineering, machine learning, application development and documentation phases spread over the semester, as shown in the Gantt chart of Figure 3.1. Parallel work on the dashboard and the models made it possible to complete the project well within the academic schedule.



**Figure 3.1:** Project schedule (Gantt chart)

## 3.5 Cost Estimation

The software cost is estimated with the **Basic COCOMO** model [26] in *organic* mode, which suits a small team working on a well-understood problem. The implemented source code (Python, JavaScript and CSS, excluding blank lines and third-party libraries) measures approximately **4.54 KLOC**.

$$\text{Effort } E = a \cdot (\text{KLOC})^b = 2.4 \times (4.54)^{1.05} = 11.75 \text{ person-months} \quad (3.1)$$

$$\text{Development time } T = c \cdot E^d = 2.5 \times (11.75)^{0.38} = 6.38 \text{ months} \quad (3.2)$$

$$\text{Average staffing } S = E / T = 1.84 \text{ persons} \quad (3.3)$$

The three-member team is consistent with the estimated average staffing of about 1.8 persons. Table 3.3 converts the effort into a notional cost, assuming the cost of a fresh software engineer, and lists the actual expenditure incurred by the team.

**Table 3.3:** Cost estimation

| Item               | Basis                         | Notional cost (₹) | Actual cost (₹)     |
|--------------------|-------------------------------|-------------------|---------------------|
| Development effort | 11.75 person-months × ₹40,000 | 4,69,980          | 0 (student project) |

| Item                     | Basis                                                  | Notional cost (₹) | Actual cost (₹) |
|--------------------------|--------------------------------------------------------|-------------------|-----------------|
| Hardware                 | Existing student laptops                               | 0                 | 0               |
| Software licences        | Spark, MongoDB, Python, FastAPI, ECharts – open source | 0                 | 0               |
| Cloud hosting (optional) | MongoDB Atlas free tier                                | 0                 | 0               |
| Internet & electricity   | Approximate, for the project duration                  | 3,000             | 3,000           |
| Documentation & printing | Report binding and printing                            | 1,500             | 1,500           |
| Total                    |                                                        | 4,74,480          | 4,500           |

### 3.6 Functional Requirements

**Table 3.4:** Functional requirements

| ID   | Requirement                                                                                                                                                        |
|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FR1  | The system shall generate a synthetic dataset of configurable size (small, medium, large) or accept raw files with the same schema.                                |
| FR2  | The system shall ingest CSV and JSON-lines files with explicit schemas using Apache Spark.                                                                         |
| FR3  | The system shall remove duplicates, standardise categorical values, handle missing values, discard invalid records and report the number of corrections per table. |
| FR4  | The system shall compute KPIs, daily/monthly/category/geographic sales, payment and device breakdowns, sale-event uplift and product statistics.                   |
| FR5  | The system shall sessionise click-stream events and compute conversion funnels, engagement metrics and cohort retention.                                           |
| FR6  | The system shall segment customers using K-Means on RFM features and select k automatically.                                                                       |
| FR7  | The system shall predict each customer's probability of churning in the next 90 days and explain the prediction.                                                   |
| FR8  | The system shall recommend ten products per customer and list similar products for every product.                                                                  |
| FR9  | The system shall mine association rules at product and sub-category level.                                                                                         |
| FR10 | The system shall forecast revenue for the next 60 days with prediction intervals and detect anomalous days and orders.                                             |
| FR11 | The system shall store all results in MongoDB and expose them through a REST API.                                                                                  |
| FR12 | The dashboard shall provide filters, customer search, a what-if churn simulator and CSV export of campaign lists.                                                  |

| ID   | Requirement                                                                                        |
|------|----------------------------------------------------------------------------------------------------|
| FR13 | The system shall log every pipeline run with stage timings, data-quality report and model metrics. |

### 3.7 Non-Functional Requirements

- **Performance:** the medium dataset ( $\approx 1.4$  million records) shall be processed end-to-end in under 10 minutes on a quad-core laptop; dashboard API responses shall be returned in under 300 ms.
- **Scalability:** the same code shall run on a Spark cluster and a MongoDB replica set by changing configuration only.
- **Reliability:** the pipeline shall validate the MongoDB connection before starting, record failures in the run log and be re-runnable (idempotent overwrite of collections).
- **Usability:** the dashboard shall be responsive, support light and dark themes, use Indian number formatting and require no training for basic use.
- **Portability:** the system shall run on Windows, macOS and Linux with Python 3.11 and Java 17.
- **Maintainability:** code shall be modular (data generation, ETL, analytics, ML, API, UI) and covered by unit tests.
- **Reproducibility:** all random processes shall use fixed seeds so that results can be reproduced exactly.
- **Privacy:** no real personal data shall be used; all names, e-mail addresses and brands are synthetic.

### 3.8 Dataset Specification

The default *medium* dataset simulates two years (2024-07-01 to 2026-06-30) of an Indian multi-category marketplace. Table 3.5 lists the raw files and their volumes.

**Table 3.5:** Raw dataset specification (medium scale)

| File          | Format | Records | Key attributes                                                                              |
|---------------|--------|---------|---------------------------------------------------------------------------------------------|
| customers.csv | CSV    | 10,033  | customer_id, name, gender, age, city, state, region, tier, signup date, acquisition channel |

| File            | Format                           | Records   | Key attributes                                                                          |
|-----------------|----------------------------------|-----------|-----------------------------------------------------------------------------------------|
| products.jsonl  | JSON lines                       | 560       | product_id, name, category (8), sub-category (35), brand, list and cost price           |
| orders.csv      | CSV                              | 74,348    | order_id, customer_id, timestamp, status, payment method, device, coupon, shipping fee  |
| order_items.csv | CSV                              | 1,45,368  | order_id, line_no, product_id, quantity, list and unit price, discount, rating          |
| events/*.jsonl  | JSON lines<br>(24 monthly files) | 13,51,601 | event_id, session_id, customer_id, time, event type, product_id, device, traffic source |

The generator models six hidden customer personas (loyal, regular, bargain hunter, occasional, one-time and browser) with different purchase rates, basket sizes, coupon usage and lifetimes. Demand follows weekends, salary days and nine Indian sale events; engagement decays in the sixty days before a customer churns; eighty percent of products have a companion product that is frequently bought with them; and four anomalous days are planted as ground truth. Roughly half a percent of dirty records are injected deliberately so that the cleaning stage can be evaluated.

## CHAPTER 4

### PROJECT ANALYSIS & DESIGN

#### 4.1 Introduction

System analysis identifies *who* uses the system and *what* it must do; system design decides *how* the requirements are realised. This chapter models ShopSense with UML diagrams (use-case, sequence, state, activity and class diagrams), data-flow diagrams at two levels, the overall architecture, the MongoDB database design, the deployment view and the user-interface structure. All diagrams were produced from text definitions (Mermaid) stored in the project repository so that they stay synchronised with the code.

#### 4.2 UML Diagrams

##### 4.2.1 Use-case diagram

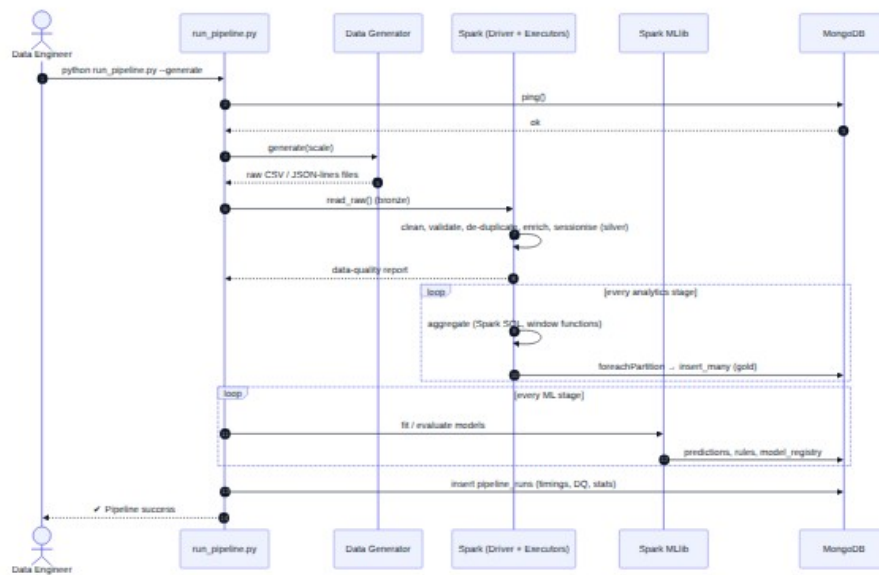
The use-case diagram in Figure 4.1 shows the four actors that interact with ShopSense and the goals they achieve. The **Business Analyst** monitors KPIs, analyses sales and funnels, studies forecasts and anomalies and searches customer profiles. The **Marketing Manager** works with the machine-learning outputs: customer segments, churn predictions and the what-if simulator, win-back list export, recommendations and bundle rules. The **Data Engineer** generates or ingests data, runs the Spark pipeline and monitors data quality and model metrics. External systems such as a CRM or BI tool consume the REST API. The *Run Spark pipeline* use case includes *Generate / ingest data*, and *Run what-if simulation* extends *Predict & explain churn*.



**Figure 4.1:** Use-case diagram of ShopSense

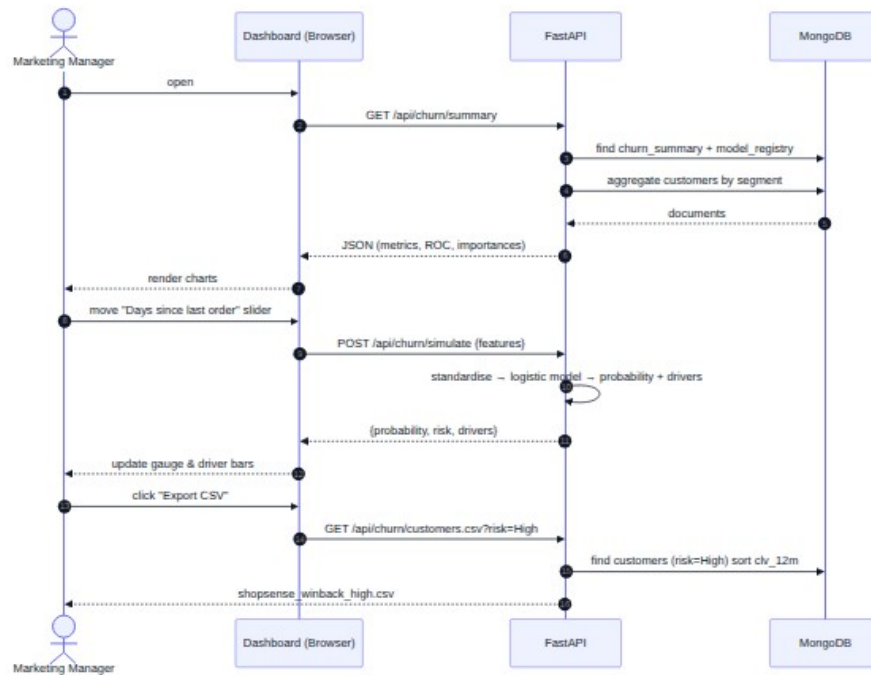
#### 4.2.2 Sequence diagrams

Figure 4.2 describes the interaction between the objects during a pipeline run. The data engineer launches *run\_pipeline.py*, which first checks that MongoDB is reachable and, if required, invokes the data generator. Spark then reads the raw files (bronze), cleans and enriches them (silver) and returns a data-quality report. For every analytics stage Spark aggregates the silver DataFrames and each partition writes its documents to MongoDB in parallel; for every machine-learning stage MLlib fits and evaluates the models and writes predictions, rules and registry entries. Finally a run document containing all stage timings is stored.



**Figure 4.2:** Sequence diagram: execution of the Spark pipeline

Figure 4.3 shows a typical interactive session. When the marketing manager opens the churn page, the browser calls the REST API, which reads the churn summary and model registry from MongoDB and aggregates customers by segment. When a slider in the what-if simulator is moved, the browser posts the feature values; the API standardises them, applies the exported logistic-regression coefficients and returns the probability with its main drivers in about a millisecond, without starting Spark. The *Export CSV* action streams a campaign file of high-risk customers.

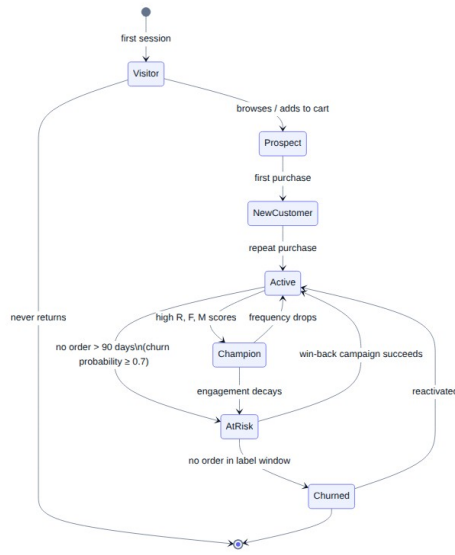


**Figure 4.3:** Sequence diagram: churn dashboard and what-if simulator

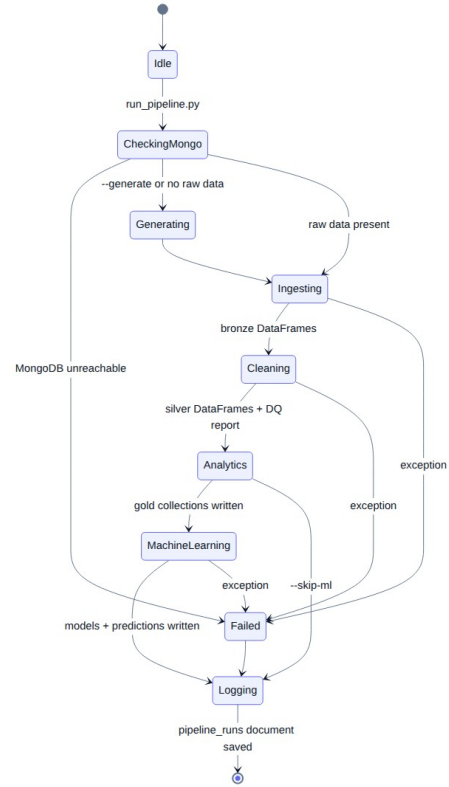
### 4.2.3 State diagrams

Figure 4.4(a) shows the customer life-cycle modelled by the analytics. A visitor becomes a prospect when they browse and add items to the cart, and a new customer after the first purchase. Repeat purchases make the customer *active*; high recency, frequency and monetary scores make them a *champion*. When no order is placed for more than ninety days the customer moves to *at risk*, from where a successful win-back campaign returns them to *active*; otherwise they become *churned*. The segmentation and churn models are designed to detect exactly these transitions.

Figure 4.4(b) shows the states of a pipeline run. After verifying the MongoDB connection the run optionally generates data, then passes through ingestion, cleaning, analytics and machine learning. Any exception moves the run to the *failed* state; in every case the run finishes by logging a *pipeline\_runs* document, so failures are always visible on the dashboard.



(a) Customer life-cycle



(b) Pipeline run

**Figure 4.4:** State diagrams

#### 4.2.4 Activity diagram

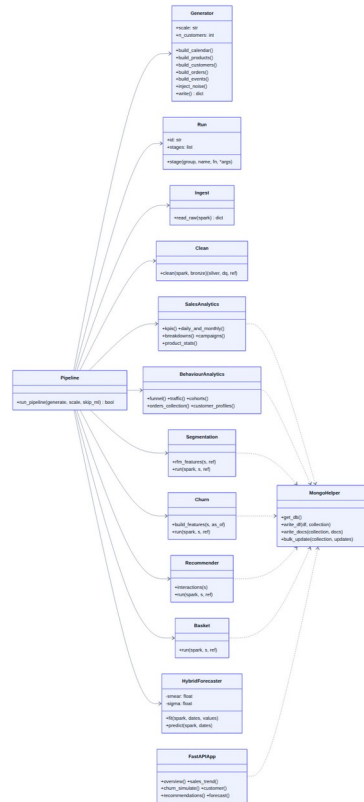
The activity diagram in Figure 4.5 details the churn-prediction workflow, the most involved machine-learning process. After the cutoff date is computed, twenty-one behavioural features are built only from data before the cutoff and the label is derived from the following ninety days. Three models are trained in parallel branches and evaluated on the same held-out test set; the model with the highest ROC-AUC re-scores all customers as of today, risk bands and customer value are computed, the explainable logistic model is exported and all results are written to MongoDB.



**Figure 4.5:** Activity diagram: churn prediction workflow

#### 4.2.5 Class diagram

Figure 4.6 presents the main modules of the implementation as classes. The *Pipeline* orchestrator uses a *Run* object to time every stage and depends on the *Generator*, the *Ingest* and *Clean* modules, the two analytics modules and the five machine-learning modules (*Segmentation*, *Churn*, *Recommender*, *Basket* and the *HybridForecaster*). All modules write through the *MongoHelper*, which is also used by the *FastAPI* application to serve the dashboard, giving a clean separation between processing and serving.



**Figure 4.6:** Class diagram of the ShopSense modules

#### 4.2.6 Data flow diagrams

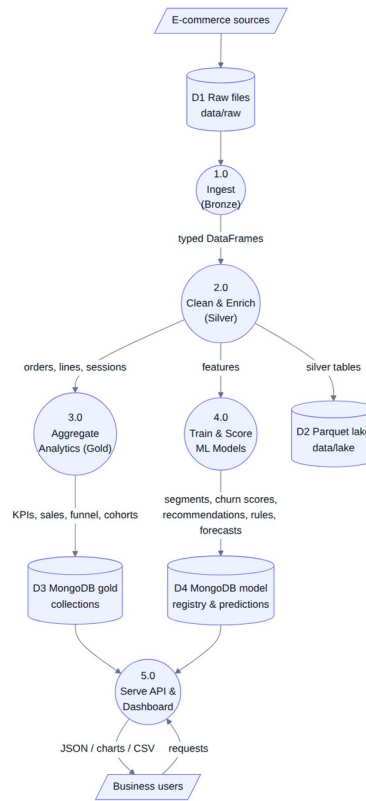
The context-level (level-0) DFD in Figure 4.7 treats ShopSense as a single process. The e-commerce platform supplies raw orders, products, customers and events; the data engineer triggers runs and receives the run log and data-quality report; business users send filters, searches and what-if inputs and receive dashboards, forecasts, segments, churn lists, recommendations.



**Figure 4.7:** Data flow diagram – level 0 (context diagram)

The level-1 DFD in Figure 4.8 decomposes the system into five processes and four data stores. Process 1.0 ingests the raw files (D1) into typed DataFrames; process 2.0 cleans

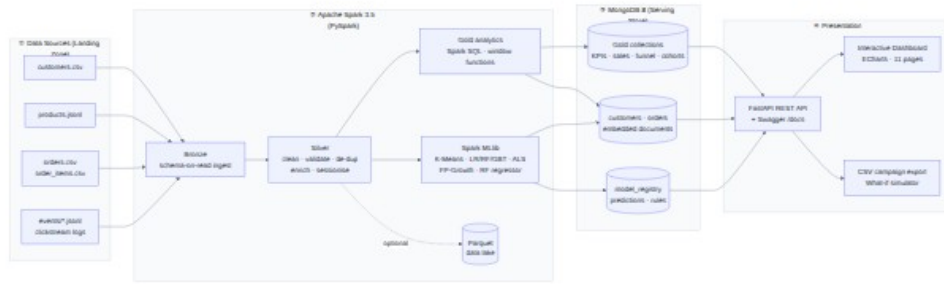
and enriches them and optionally persists the silver tables to the Parquet lake (D2); process 3.0 aggregates the analytics tables into MongoDB (D3); process 4.0 trains and scores the models and writes predictions and the model registry (D4); process 5.0 serves the API and dashboard to business users.



**Figure 4.8:** Data flow diagram – level 1

### 4.3 System Architecture

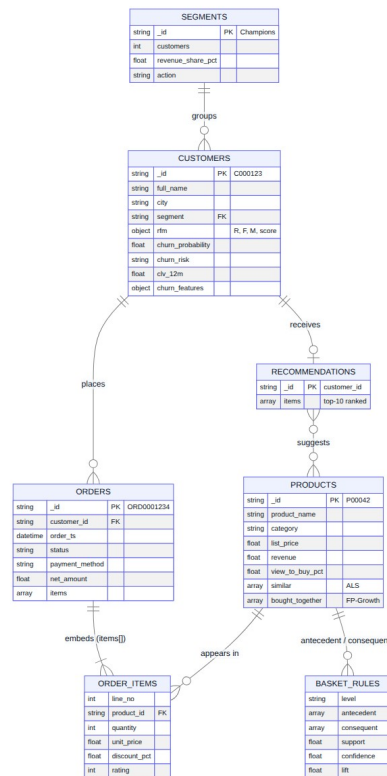
The overall architecture (Figure 4.9) has four layers. (1) The **landing zone** contains raw files exactly as a platform would export them. (2) The **processing layer** is Apache Spark: bronze ingestion, silver cleaning and enrichment, gold analytics with Spark SQL and model training with MLlib. (3) The **serving layer** is MongoDB, which stores small, query-shaped gold documents, customer and order documents with embedded data, and the model registry. (4) The **presentation layer** is a FastAPI REST service and a single-page dashboard built with Apache ECharts. Heavy computation happens only in the batch layer, while every user interaction is answered from indexed MongoDB documents; this separation mirrors the batch/serving split of large-scale analytics architectures.



**Figure 4.9:** System architecture of ShopSense

## 4.4 Database Design

MongoDB was chosen for the serving layer because analytics results are naturally hierarchical and heterogeneous: a customer profile holds an RFM sub-document, churn features and predictions; an order holds a variable number of line items; a product holds lists of similar and frequently-bought-together products. Figure 4.10 shows the logical entity-relationship model of the main collections. Three design principles were followed: (i) *query-shaped documents*, so that each dashboard widget reads a single small collection without joins; (ii) *embedding* of data that is always read together (order items inside orders; *similar* and *bought\_together* inside products); and (iii) *natural keys* as the `_id` field (for example C000123, P00042 and ORD0001234) for direct look-ups, with secondary indexes on every filter and sort field.



**Figure 4.10:** Logical data model of the main MongoDB collections

Table 4.1 lists the collections produced by the pipeline together with their grain and the module that writes them.

**Table 4.1:** MongoDB collections produced by the pipeline

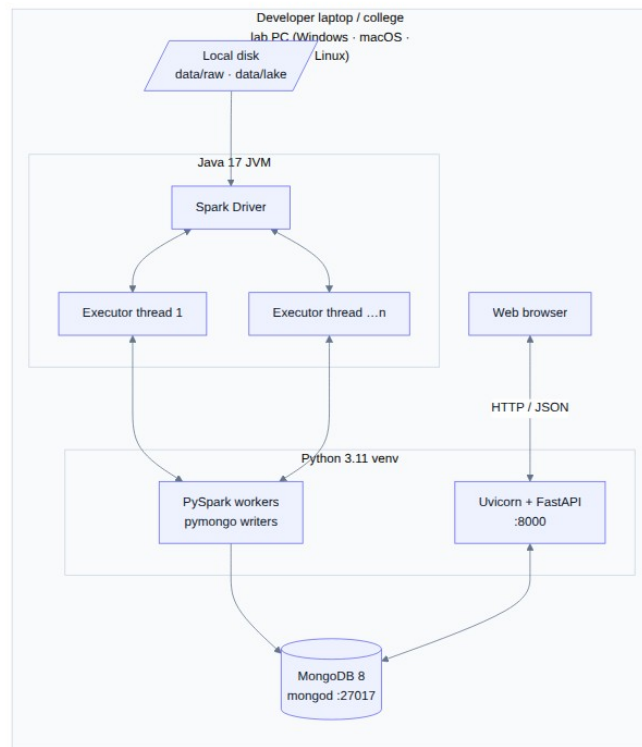
| Collection                        | Grain / content                                | Documents | Written by                     |
|-----------------------------------|------------------------------------------------|-----------|--------------------------------|
| kpis                              | Headline KPIs (single document)                | 1         | analytics.sales                |
| sales_daily                       | Revenue, orders, customers per day             | 730       | analytics.sales                |
| sales_daily_category              | Day × category (for live filters)              | 5,840     | analytics.sales                |
| sales_monthly                     | Month with new/returning customers, MoM growth | 24        | analytics.sales                |
| sales_by_category / _subcategory  | Category economics                             | 8 / 35    | analytics.sales                |
| sales_by_city / _state            | Geographic revenue                             | 28 / 20   | analytics.sales                |
| sales_by_dimension, sales_heatmap | Payment, device, tier, channel; day × hour     | 25 / 168  | analytics.sales                |
| campaigns                         | Order and revenue uplift per sale event        | 18        | analytics.sales                |
| products                          | Product KPIs + similar + bought_together       | 560       | analytics, recommender, basket |

| Collection                      | Grain / content                                 | Documents   | Written by                     |
|---------------------------------|-------------------------------------------------|-------------|--------------------------------|
| funnel, traffic_*, event_counts | Funnels, sessions by hour/month, event mix      | 10 / 48 / 7 | analytics.behaviour            |
| cohorts                         | Monthly retention per first-purchase cohort     | 24          | analytics.behaviour            |
| orders                          | Order with embedded line items                  | 74,009      | analytics.behaviour            |
| customers                       | Customer 360: profile, RFM, segment, churn, CLV | 10,000      | behaviour, segmentation, churn |
| segments, segment_points        | Segment profiles; PCA sample                    | 6 / 2,473   | ml.segmentation                |
| churn_summary                   | Model comparison, ROC, importances              | 1           | ml.churn                       |
| recommendations                 | Top-10 products per customer                    | 7,420       | ml.recommender                 |
| basket_rules                    | Association rules                               | 281         | ml.basket                      |
| forecasts, anomalies            | Forecast per series; detected anomalies         | 9 / 49      | ml.forecast                    |
| model_registry                  | Model metadata, metrics, exported coefficients  | 9           | all ML modules                 |
| pipeline_runs                   | Run log, stage timings, data-quality report     | 1+          | pipeline                       |

An example *orders* document with its embedded line items, as stored by the pipeline, is shown in Figure 6.2; embedding allows the Customer 360 page to display an order and all its items with a single read.

## 4.5 Deployment Design

Figure 4.11 shows the deployment view. On a single machine the Spark driver and executor threads run inside a Java 17 virtual machine, Python worker processes (from the project's virtual environment) execute the PyMongo partition writers, MongoDB runs as a local service on port 27017 and Uvicorn serves the FastAPI application on port 8000 to the browser. Because all addresses and resources are configured through a *.env* file, the same components can be placed on a Spark cluster, a MongoDB replica set or MongoDB Atlas and a separate web server without changing the code.



**Figure 4.11:** Deployment diagram

## 4.6 User Interface Design

The dashboard is a single-page application with a fixed sidebar (grouped into *Analytics*, *Machine Learning* and *Explore*), a top bar with global customer search, API documentation link and theme toggle, and a content area. Each page is organised as a grid of cards; each card answers one question and contains a chart or table with a short explanatory subtitle. The eleven pages cover overview, sales, products, behaviour, segmentation, churn, recommendations, market basket, forecasting, Customer 360 and pipeline monitoring; each is described with a screenshot in Section 6.2.

# CHAPTER 5

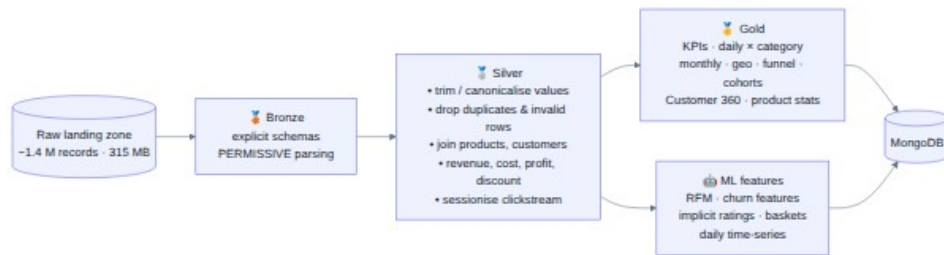
## METHODOLOGY

### 5.1 Introduction

This chapter explains how ShopSense transforms raw data into insight. It first presents the system block diagram and then describes, step by step, the data-generation model, the ingestion and cleaning algorithm, the descriptive analytics and each machine-learning technique together with its mathematical formulation and evaluation measure.

### 5.2 System Block Diagram

Figure 5.1 is the block diagram of the data flow. Approximately 1.4 million raw records enter the **bronze** block, where they are read with explicit schemas. The **silver** block standardises values, removes duplicates and invalid rows, joins orders with products and customers, derives revenue, cost, profit and discount for every line item and groups click-stream events into sessions. From the silver layer two branches start: the **gold** block computes descriptive analytics and the **ML features** block builds the inputs of the models (RFM vectors, churn features, implicit ratings, baskets and daily time series). Both branches write their outputs to MongoDB.



**Figure 5.1:** System block diagram (bronze – silver – gold data flow)

### 5.3 Data Generation Model

Each customer  $c$  is assigned a hidden persona that determines a monthly order rate  $\lambda_c$ , an average number of extra items per basket, a coupon probability, a premium-price affinity and a mean lifetime. Customers are active between their sign-up day and an

exponentially distributed end day. The number of orders of customer  $c$  on day  $d$  is drawn from a Poisson distribution:

$$N(c,d) \sim \text{Poisson}(\lambda c / 30 \cdot m(d) \cdot a(c,d) \cdot \delta(c,d)) \quad (5.1)$$

where  $m(d)$  is the daily demand multiplier,  $a(c,d) \in \{0,1\}$  indicates that the customer is active and  $\delta(c,d) = 0.45$  during the sixty days before the customer churns (pre-churn engagement decay) and 1 otherwise. The demand multiplier combines trend, weekly pattern, salary days, sale events and noise:

$$m(d) = (1 + 0.38 \cdot d/730) \cdot w(d) \cdot s(d) \cdot \prod e(d) \cdot \varepsilon(d), \quad \varepsilon \sim \text{LogNormal}(0, 0.07) \quad (5.2)$$

with  $w(d) = 1.18$  on weekends,  $s(d) = 1.10$  on the 1st–5th of a month and  $e(d)$  the boost of an active sale event, decaying linearly over the event. Products within a sub-category follow a Zipf-like popularity  $1/r^{0.9}$  and cheaper sub-categories are chosen more frequently. A purchase of a product triggers the purchase of its companion product with probability 0.38, which creates genuine co-purchase patterns. Click-stream sessions are generated for every order and for browsing visits that drop out of the funnel. Finally, four anomalous days and about 0.5% dirty records are injected.

## 5.4 Data Ingestion and Cleaning

The cleaning stage is implemented with Spark DataFrame transformations so that it scales linearly with the data. The algorithm is:

1. Read every raw file with an explicit StructType schema (PERMISSIVE mode); read the whole events folder in parallel.
2. Customers: canonicalise city names against the master list (trim, case-insensitive match), fill missing cities with “Unknown”, drop duplicate customer\_id values.
3. Products: drop duplicates and non-positive prices; derive margin = 1 – cost/list price.
4. Orders: map payment-method variants to canonical values, normalise status, drop duplicate order\_id values and rows without timestamp or customer.
5. Order items: remove rows with quantity  $\leq 0$  or unit price  $\leq 0$ ; drop duplicate (order\_id, line\_no).

6. Events: convert blank strings to null, lower-case event types, drop duplicate event\_id values, discard unknown event types and product views without a product.
7. Enrich line items: join orders and products; compute revenue = quantity × unit price, gross value, cost, profit and discount amount; flag revenue-bearing lines (status not Cancelled/Returned).
8. Aggregate lines into order totals and join customer geography; sessionise events by session\_id (start, end, duration, counts of views, carts, checkouts, purchases, bounce flag).
9. Record raw and clean counts and every correction in the data-quality report; optionally persist silver tables as Parquet.

## 5.5 Descriptive Analytics

The gold layer computes business metrics with Spark SQL aggregations and window functions. The main definitions are:

$$\text{Net revenue} = \sum (\text{quantity} \times \text{unit price}) \text{ over orders not cancelled or returned} \quad (5.3)$$

$$\text{AOV} = \text{Net revenue} / \text{Number of successful orders} \quad (5.4)$$

$$\text{Conversion rate} = \text{Sessions with a purchase} / \text{All sessions} \times 100 \quad (5.5)$$

$$\text{Cart abandonment} = (\text{Sessions with cart} - \text{Sessions with cart and purchase}) / \text{Sessions with cart} \times 100 \quad (5.6)$$

$$\text{Retention}(\text{cohort}, k) = |\text{customers of cohort buying in month } k| / |\text{cohort}| \times 100 \quad (5.7)$$

$$\text{Uplift}(\text{event}) = (\text{mean daily value during event} / \text{mean daily value of the 28 preceding non-sale days} - 1) \times 100 \quad (5.8)$$

Month-on-month growth uses the *lag* window function over the ordered months, the first-purchase month used by cohorts and by new-versus-returning analysis uses a *min* window partitioned by customer, favourite category and preferred device use *row\_number* windows, and RFM quintiles use the *ntile(5)* window. The sale-event uplift is measured for both order volume and revenue, because discounts can make revenue uplift much smaller than demand uplift.

## 5.6 Customer Segmentation using K-Means

For every purchasing customer seven features are computed as of the reference date: recency, frequency, monetary value, average order value, tenure, category diversity and average discount. Skewed variables (recency, frequency, monetary, AOV) are transformed with  $\log(1 + x)$  and all features are standardised:

$$z = (x - \mu) / \sigma \quad (5.9)$$

K-Means partitions the  $n$  standardised vectors into  $k$  clusters  $C_1 \dots C_k$  by minimising the within-cluster sum of squares:

$$J = \sum_k \sum_{x \in C_k} \|x - \mu_k\|^2 \quad (5.10)$$

Spark initialises the centres with k-means++ [9] and alternates assignment and update steps until convergence (at most 40 iterations). The number of clusters is selected with the mean silhouette coefficient [10], where  $a(i)$  is the mean distance of point  $i$  to its own cluster and  $b(i)$  the mean distance to the nearest other cluster:

$$s(i) = (b(i) - a(i)) / \max\{a(i), b(i)\}, \quad S = (1/n) \sum s(i) \quad (5.11)$$

Models are trained for  $k = 3 \dots 8$  and the best silhouette among the business-interpretable values  $k = 4 \dots 6$  is chosen. Clusters are named automatically from their centroids using the rules in Table 5.1, and a 2-D PCA projection is computed for visualisation.

**Table 5.1:** Rules for naming clusters from their centroids

| Centroid condition                                             | Segment name (in order of value)                                          | Recommended action                    |
|----------------------------------------------------------------|---------------------------------------------------------------------------|---------------------------------------|
| Recency $\leq 90$ days, tenure $\geq 150$ days                 | Champions $\rightarrow$ Loyal Customers $\rightarrow$ Potential Loyalists | Reward, up-sell, loyalty programme    |
| Recency $\leq 90$ days, tenure $< 150$ days, frequency $< 2.5$ | New & Promising                                                           | Onboarding and second-purchase offers |
| $90 < \text{Recency} \leq 240$ days                            | At Risk (frequency $\geq 2.5$ ) / Needs Attention / About to Sleep        | Win-back and time-limited offers      |
| Recency $> 240$ days                                           | Can't Lose Them (frequency $\geq 4$ ) / Hibernating / Lost                | VIP outreach or low-cost reactivation |
| No purchase                                                    | Prospects                                                                 | First-order discount, retargeting     |

## 5.7 Churn Prediction

### 5.7.1 Problem formulation

Churn is defined operationally: a customer has churned if they place no successful order during the next  $W = 90$  days. Let  $T$  be the last date of the data. The cutoff is  $t_0 = T - W + 1$ . Features are computed only from events before  $t_0$  (observation window) and the label from  $[t_0, T]$  (label window):  $y = 1$  if the customer has no revenue-bearing order in the label window, else 0. Only customers with at least one purchase before  $t_0$  are included (5,484 customers, of whom 57.17% churned). This time-based design prevents information from the future leaking into the features.

### 5.7.2 Feature engineering

**Table 5.2:** Churn prediction features

| Group                          | Features                                                                                       |
|--------------------------------|------------------------------------------------------------------------------------------------|
| Recency / frequency / monetary | days since last order, number of orders, total spend, average order value, tenure              |
| Trend                          | orders in last 90 days, orders in previous 90 days, spend in last 90 days                      |
| Engagement (click-stream)      | sessions in last 30 and 90 days, days since last visit, cart-abandonment rate (last 180 days)  |
| Behaviour                      | average discount %, return rate, cancellation rate, category diversity, cash-on-delivery share |
| Profile                        | age, city tier, preferred device (one-hot), acquisition channel (one-hot)                      |

### 5.7.3 Models

**Logistic Regression** models the probability of churn with the sigmoid of a linear combination of standardised features and is trained by minimising the regularised log-loss; the regularisation strength  $\lambda$  is selected by 3-fold cross-validation over  $\{0.001, 0.01, 0.1\}$ :

$$p(y = 1 | z) = \sigma(\beta_0 + \sum \beta_i z_i), \quad \sigma(t) = 1 / (1 + e^{-(t)}) \quad (5.12)$$

$$\min_{\beta} - \sum [y \log p + (1 - y) \log(1 - p)] + \lambda \|\beta\|^2 \quad (5.13)$$

**Random Forest** [12] trains 150 decision trees of depth 8 on bootstrap samples, considering a random subset of  $\sqrt{d}$  features at each split, and averages their class probabilities. **Gradient-Boosted Trees** [13] fit 50 trees of depth 5 sequentially, each

tree fitting the negative gradient of the log-loss of the current ensemble with learning rate 0.1:

$$F_{-m}(x) = F_{-(m-1)}(x) + \eta \cdot h_{-m}(x), \quad h_{-m} \approx -\partial L(y, F) / \partial F \quad (5.14)$$

#### 5.7.4 Evaluation measures

The data is split randomly into 80% training and 20% test customers. With TP, FP, TN and FN denoting the confusion-matrix counts at threshold 0.5:

$$Precision = TP / (TP + FP), \quad Recall = TP / (TP + FN), \quad F1 = 2 \cdot P \cdot R / (P + R) \quad (5.15)$$

$$Accuracy = (TP + TN) / (TP + TN + FP + FN) \quad (5.16)$$

The ROC curve plots the true-positive rate against the false-positive rate for all thresholds; its area (AUC) equals the probability that a randomly chosen churning customer receives a higher score than a randomly chosen non-churning customer [14]. The model with the highest AUC re-scores every customer with features as of  $T + 1$ , customers are banded into High ( $p \geq 0.7$ ), Medium ( $0.4 \leq p < 0.7$ ) and Low risk, and a simple predicted 12-month customer value is computed:

$$CLV_{12} = (Monetary / \max(tenure\ months, 1)) \times 12 \times (1 - p_{churn}) \quad (5.17)$$

For explainability the logistic model's coefficients and the scaler's mean and standard deviation are exported to MongoDB. The contribution of feature  $i$  to a customer's log-odds is  $\beta_i \cdot z_i$ ; the largest positive and negative contributions are displayed as the "drivers" of the prediction and power the what-if simulator.

## 5.8 Product Recommendation using ALS

Implicit feedback is converted to an interaction strength  $r(u, i)$  for every customer  $u$  and product  $i$  and capped at 30:

$$r(u, i) = 5 \cdot purchases + 2 \cdot add-to-cart + 1.5 \cdot wishlist + 0.5 \cdot views \quad (5.18)$$

Following Hu, Koren and Volinsky [16], the preference  $p(u, i) = 1$  if  $r(u, i) > 0$  (else 0) is weighted by a confidence  $c(u, i) = 1 + \alpha \cdot r(u, i)$ , and user factors  $x_u$  and item factors  $y_i$  of rank  $f$  are found by minimising:

$$\min_{x, y} \sum_{(u, i)} c(u, i) (p(u, i) - x_u^T y_i)^2 + \lambda (\sum_u \|x_u\|^2 + \sum_i \|y_i\|^2) \quad (5.19)$$

ALS alternates closed-form least-squares solutions for all user vectors (keeping item vectors fixed) and for all item vectors, which Spark distributes across partitions. The model is evaluated with a *leave-last-purchase-out* protocol: for customers with at least three purchased products the most recent purchase is hidden, the model is trained on all remaining interactions and the Top-10 list (excluding previously bought items) is checked for the hidden item:

$$HitRate@10 = (1/|U|) \sum_u 1[hidden\ item \in Top-10(u)], \quad NDCG@10 = (1/|U|) \sum_u 1 / \log_2(rank + 1) \quad (5.20)$$

A popularity recommender is evaluated in the same way as a baseline, and a grid over rank,  $\lambda$  and  $\alpha$  selects the final configuration. Similar products are obtained from the cosine similarity of item-factor vectors.

## 5.9 Market-Basket Analysis using FP-Growth

Every non-cancelled order is a transaction (set of products; separately, set of sub-categories). For a rule  $A \rightarrow B$ :

$$support(A \rightarrow B) = P(A \cup B), \quad confidence(A \rightarrow B) = support(A \cup B) / support(A) \quad (5.21)$$

$$lift(A \rightarrow B) = confidence(A \rightarrow B) / support(B) \quad (5.22)$$

FP-Growth [18] scans the transactions twice: the first pass counts item frequencies, the second builds a compact FP-tree in which transactions share prefixes of frequent items. Frequent itemsets are then mined recursively from conditional FP-trees without generating candidates; Spark’s PFP implementation [19] partitions this work across executors. Rules with lift  $> 1$  are stored, and the single-antecedent product rules are attached to each product as “frequently bought together”.

## 5.10 Sales Forecasting

Daily revenue is modelled on the logarithmic scale as the sum of a trend and a calendar-driven seasonal component:

$$\log(1 + y_t) = \beta_0 + \beta_1 \cdot t + g(calendar_t) + \varepsilon_t \quad (5.23)$$

The trend is fitted with Spark LinearRegression on the day index  $t$ ; the residuals are then modelled by a RandomForestRegressor  $g$  (120 trees, depth 7) using day of week, month, day of month, weekend flag, salary-day flag, days to month end and the *planned*

marketing calendar (sale-event flag and day within the event), inspired by the holiday component of Prophet [20]. Because  $\exp(E[\log y])$  underestimates  $E[y]$ , Duan’s smearing factor [24] is applied:

$$\hat{y}_t = \exp(\hat{\beta}_0 + \hat{\beta}_1 \cdot t + \hat{g}(\text{calendar}_t)) \cdot (1/n) \sum \exp(\hat{\epsilon}_i) \quad (5.24)$$

Prediction intervals use the residual standard deviation  $\hat{\sigma}$  on the log scale:  $\hat{y} \cdot \exp(\pm 1.28\hat{\sigma})$  for 80% and  $\hat{y} \cdot \exp(\pm 1.96\hat{\sigma})$  for 95%. The last 60 days are held out and the model is compared with a seasonal-naïve forecast (repeat last week), a 28-day moving average and a calendar linear regression [21] using:

$$MAPE = (100/n) \sum |y_t - \hat{y}_t| / y_t, \quad MAE = (1/n) \sum |y_t - \hat{y}_t|, \quad R^2 = 1 - \sum (y_t - \hat{y}_t)^2 / \sum (y_t - \bar{y})^2 \quad (5.25)$$

Because individual days are dominated by a few high-value baskets, MAPE is also reported on weekly totals. The same procedure is applied to the total series and to each of the eight categories, after which the model is refitted on the full history to forecast the next 60 days.

## 5.11 Anomaly Detection

Anomalies are *contextual* [22]: a spike on a festival is expected, the same spike on an ordinary Tuesday is not. The hybrid model is therefore fitted to the daily **order counts** (which are less noisy than revenue), and the residual  $r_t = \log(1 + y_t) - \log(1 + \hat{y}_t)$  is standardised with the median and the median absolute deviation (MAD), which are not distorted by the anomalies themselves [23]:

$$z_t = (r_t - \text{median}(r)) / (1.4826 \cdot \text{MAD}(r)), \quad \text{flag if } |z_t| \geq 4 \text{ (critical if } \geq 7) \quad (5.26)$$

Orders above the fence  $Q3 + 3 \cdot \text{IQR}$  and customers with at least three returns and a 40% return rate are also flagged; the detector is validated against the four planted anomalies.

## 5.12 Distributed Loading into MongoDB

Gold DataFrames are written partition-parallel: the target collection is dropped, *foreachPartition* runs a writer inside every executor task that opens a PyMongo client, converts each Row to a BSON-safe document (dates to ISO strings, decimals to floats, vectors to lists, NaN to null) and bulk-inserts batches of 2,000 unordered documents; indexes are then created on the driver. Writes therefore scale with the executors without

extra JARs; MONGO\_SPARK\_CONNECTOR=true switches to the official MongoDB Spark Connector.

# CHAPTER 6

## IMPLEMENTATION DETAILS

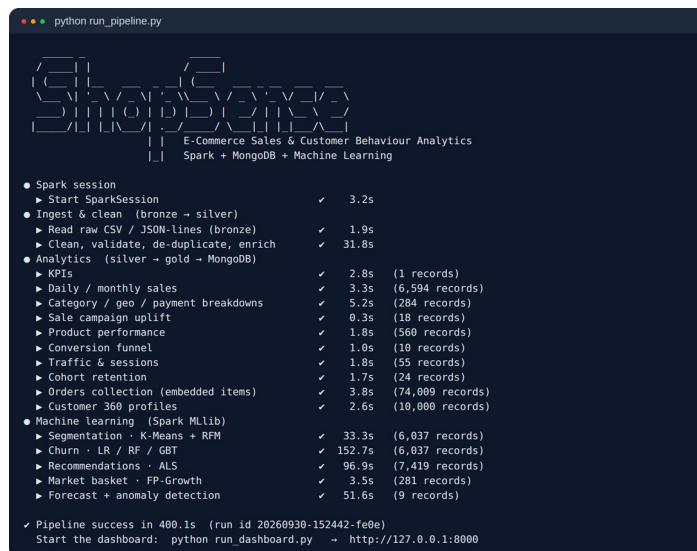
### 6.1 Introduction

ShopSense is implemented in about 4.5 thousand lines of Python, JavaScript and CSS, organised as a Python package (*shopsense*) for data generation, ETL, analytics and machine learning, a FastAPI application (*app*) with a static single-page dashboard, command-line entry points, unit tests and helper scripts. This chapter presents the implementation of every component with screenshots of the running system, the project structure, key code fragments, the REST API and the testing carried out.

### 6.2 System Implementation

#### 6.2.1 Running the Big Data pipeline

The complete pipeline is started with a single command, *python run\_pipeline.py*. Figure 6.1 shows the console output of a run on the medium dataset: every stage is timed and reports the number of documents it wrote to MongoDB. Cleaning and enrichment of the 1.35 million click-stream events and 145 thousand line items takes about 32 seconds; the complete run, including training of all models, finishes in 399 seconds.



```
python run_pipeline.py

E-Commerce Sales & Customer Behaviour Analytics
Spark + MongoDB + Machine Learning

• Spark session
  ▶ Start SparkSession ✓ 3.2s
• Ingest & clean (bronze → silver)
  ▶ Read raw CSV / JSON-lines (bronze) ✓ 1.9s
  ▶ Clean, validate, de-duplicate, enrich ✓ 31.8s
• Analytics (silver → gold → MongoDB)
  ▶ KPIs ✓ 2.8s (1 records)
  ▶ Daily / monthly sales ✓ 3.3s (6,594 records)
  ▶ Category / geo / payment breakdowns ✓ 5.2s (284 records)
  ▶ Sale campaign uplift ✓ 0.3s (18 records)
  ▶ Product performance ✓ 1.8s (560 records)
  ▶ Conversion funnel ✓ 1.8s (10 records)
  ▶ Traffic & sessions ✓ 1.8s (55 records)
  ▶ Cohort retention ✓ 1.7s (24 records)
  ▶ Orders collection (embedded items) ✓ 3.8s (74,009 records)
  ▶ Customer 360 profiles ✓ 2.6s (10,000 records)
• Machine learning (Spark MLlib)
  ▶ Segmentation · K-Means + RFM ✓ 33.3s (6,037 records)
  ▶ Churn · LR / RF / GBT ✓ 152.7s (6,037 records)
  ▶ Recommendations · ALS ✓ 96.9s (7,419 records)
  ▶ Market basket · FP-Growth ✓ 3.5s (281 records)
  ▶ Forecast + anomaly detection ✓ 51.6s (9 records)

✓ Pipeline success in 400.1s (run id 20260930-152442-fe0e)
Start the dashboard: python run_dashboard.py → http://127.0.0.1:8000
```

**Figure 6.1:** Console output of a complete pipeline run

The results are stored in the *shopsense* database. Figure 6.2 shows the collections with their document counts and an *orders* document retrieved with the MongoDB shell, in which the order line items are embedded as an array.

[illegible]

**Figure 6.2:** MongoDB collections and an embedded order document

### 6.2.2 Overview dashboard

The overview page (Figure 6.3) is the landing page for managers. A gradient banner summarises two-year net revenue (₹45.94 Cr), gross profit, the 30-day forecast, the number of high-churn-risk customers and the average product rating. Six KPI tiles compare the last 30 days with the previous 30 days and show monthly sparklines; below them are the monthly revenue chart, the category donut, new-versus-returning customers, the conversion funnel, the revenue share of each segment, the top products and automatically generated alerts.

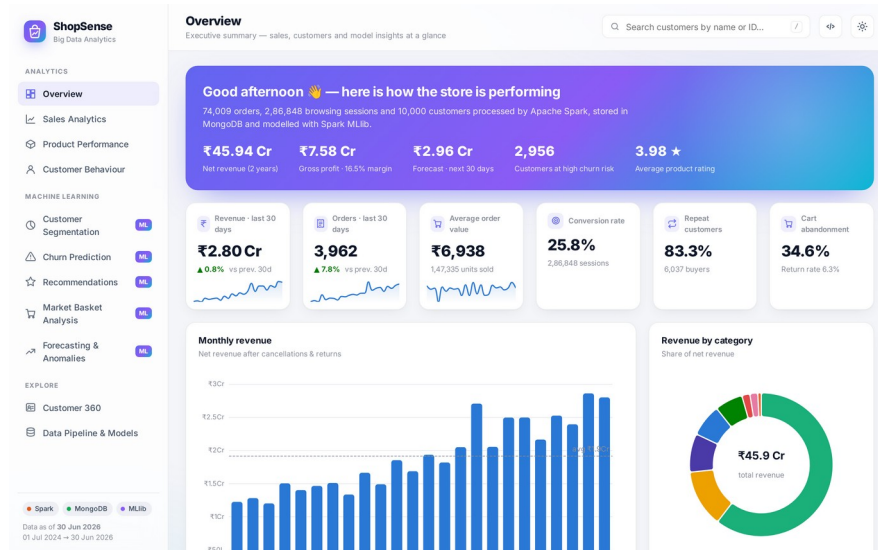


Figure 6.3: Overview dashboard

### 6.2.3 Sales analytics

The sales page (Figure 6.4) provides a filter bar with preset date ranges, a category selector and day/week/month granularity. Every change triggers a live MongoDB aggregation pipeline over the Spark-built daily  $\times$  category collection, so filters respond instantly. The page includes the revenue trend with a zoom slider, category performance and economics, a geographic bubble map, an orders heat-map by day and hour, a sub-category treemap, payment-method, device and acquisition-channel breakdowns and the sale-event uplift chart.

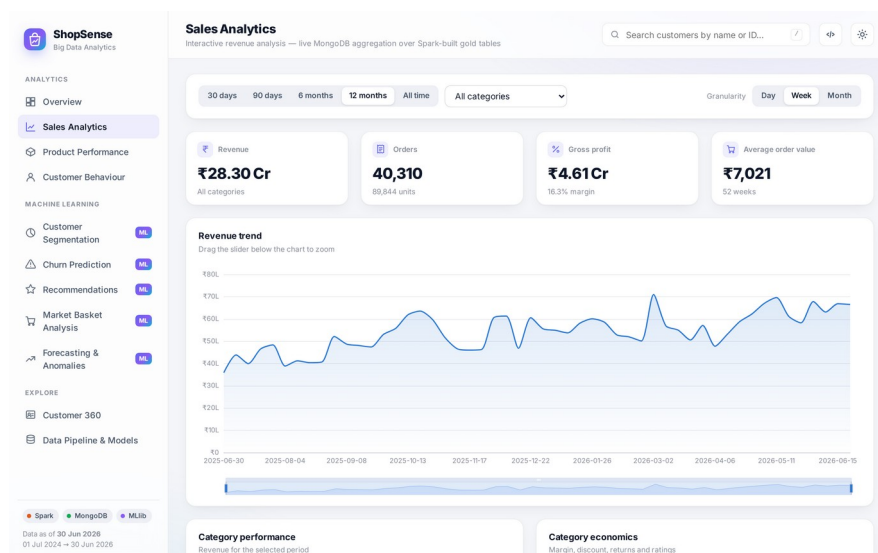


Figure 6.4: Sales analytics page with filters

## 6.2.4 Product performance

Figure 6.5 shows the product page, where the catalogue can be searched, filtered by category and sorted by revenue, units, rating, view-to-purchase conversion or return rate, next to a scatter plot of product views against orders. Clicking a product opens a detail drawer with its KPIs, the products frequently bought with it (FP-Growth) and similar products derived from the ALS item factors.

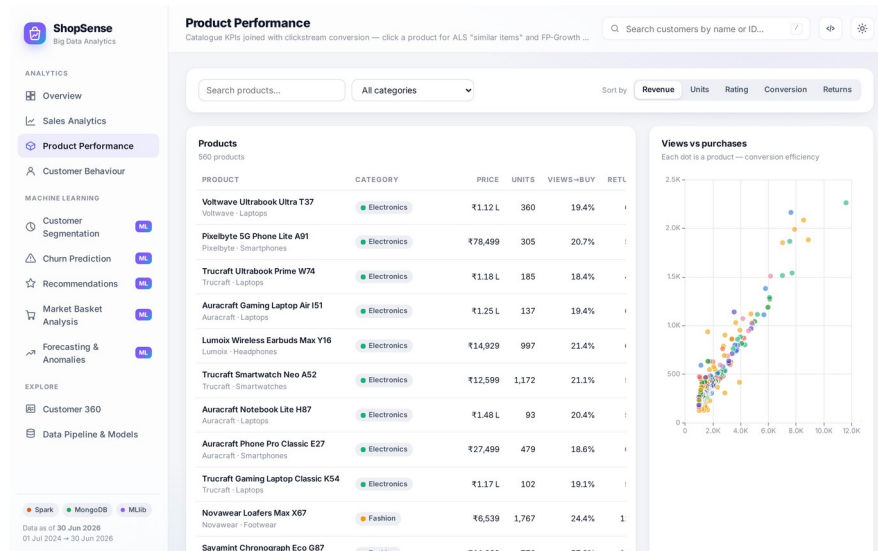


Figure 6.5: Product performance page

## 6.2.5 Customer behaviour

The behaviour page (Figure 6.6) is built from the sessionised click-stream. It shows sessions, session conversion, cart abandonment and average session duration, the conversion funnel that can be filtered by device or traffic source, conversion and bounce rates by traffic source, sessions by hour, the event mix, monthly engagement and the cohort-retention matrix.

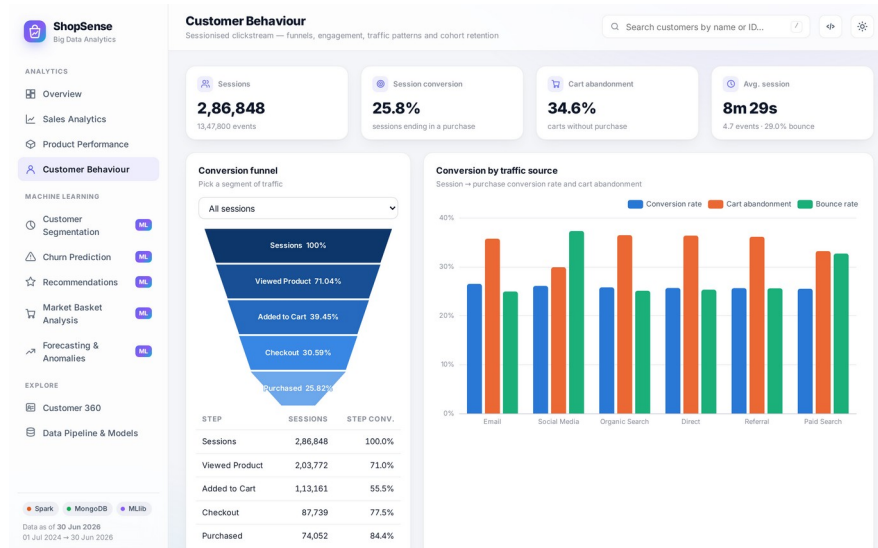


Figure 6.6: Customer behaviour page

## 6.2.6 Customer segmentation

Figure 6.7 shows the segmentation page: one card per segment with its size, revenue share, average recency, frequency and monetary value and the recommended marketing action; the PCA cluster map; the silhouette curve used to select  $k$ ; a customer-share versus revenue-share comparison; the segment profile table; and the list of customers in the selected segment, which can be exported.

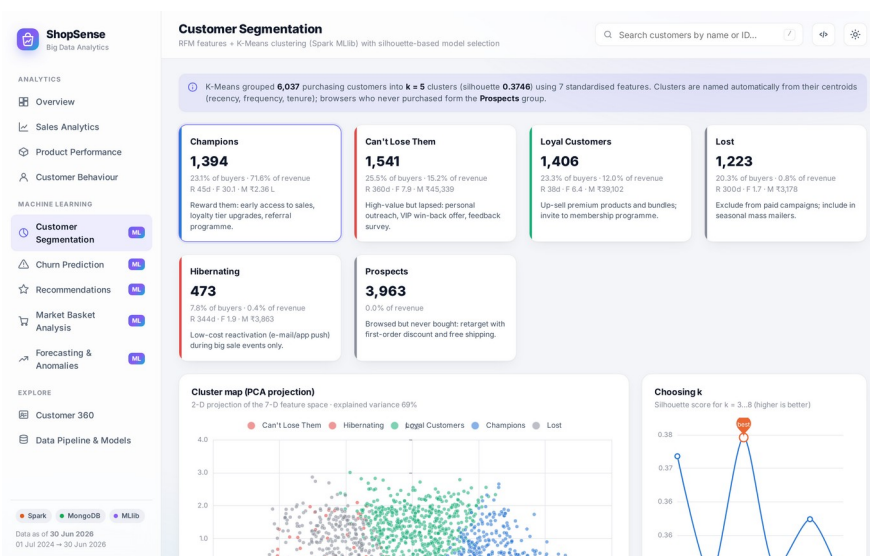


Figure 6.7: Customer segmentation page

## 6.2.7 Churn prediction and what-if simulator

The churn page (Figure 6.8) compares the three models, plots their ROC curves, shows the feature importance of the tree model and the confusion matrix of the best model, the

distribution of predicted probabilities and the average risk per segment. The **what-if simulator** (Figure 6.9) lets a marketer change a customer’s behaviour with sliders and immediately see the new probability and the features driving it. The **win-back target list** at the bottom sorts at-risk customers by predicted 12-month value and can be exported to CSV for an e-mail or CRM campaign.

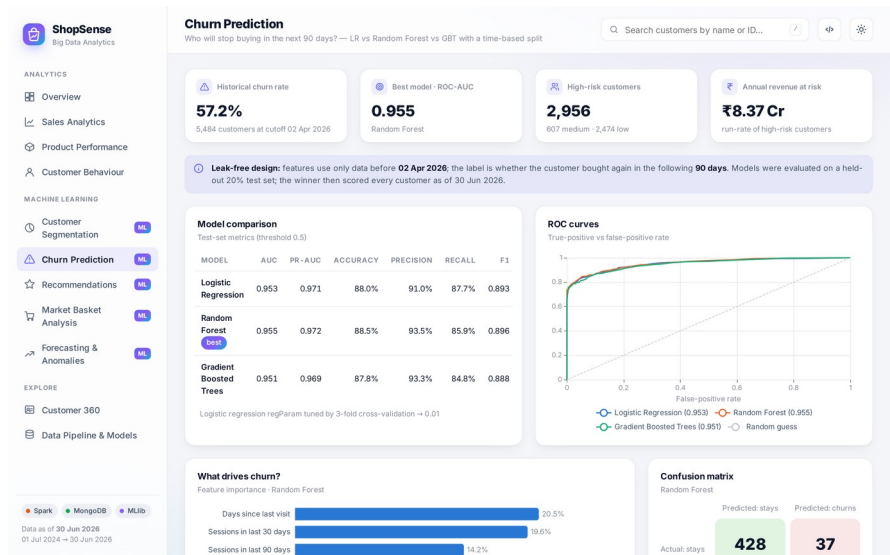


Figure 6.8: Churn prediction page

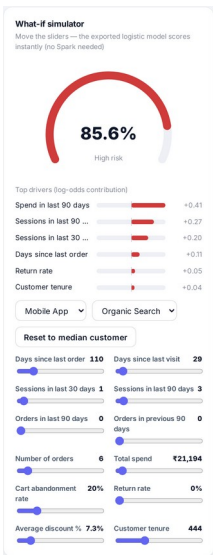


Figure 6.9: What-if churn simulator with prediction drivers

### 6.2.8 Recommendations

Figure 6.10 shows personalised recommendations for a selected customer. The customer’s recent purchases are listed first, followed by the Top-10 ALS recommendations (already-purchased products excluded), each with a relative score

and a reason. The evaluation panel compares ALS with a popularity baseline and shows the hyper-parameter grid.

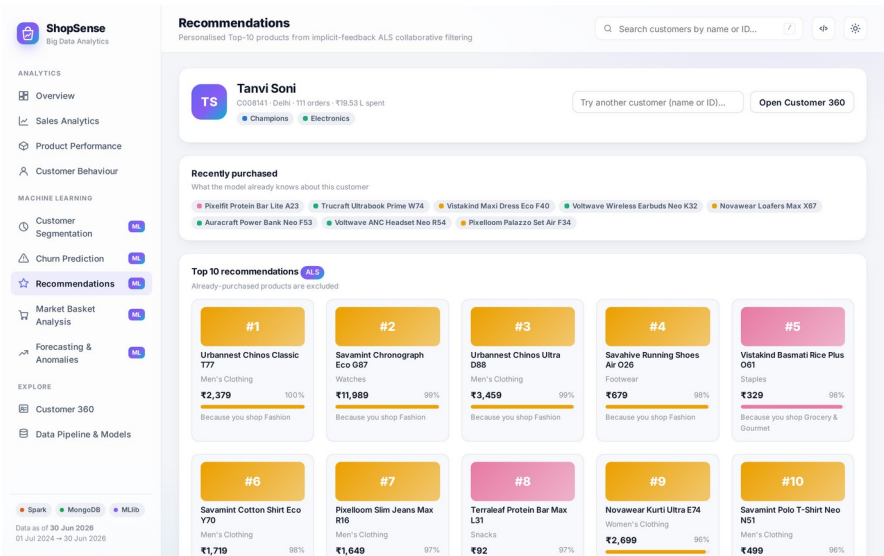


Figure 6.10: Personalised recommendations page

### 6.2.9 Market-basket analysis

The market-basket page (Figure 6.11) shows the FP-Growth statistics, an interactive force-directed network of sub-categories connected by association rules (edge width proportional to lift) and a rule explorer that can switch between product and sub-category level and filter by minimum lift or text.

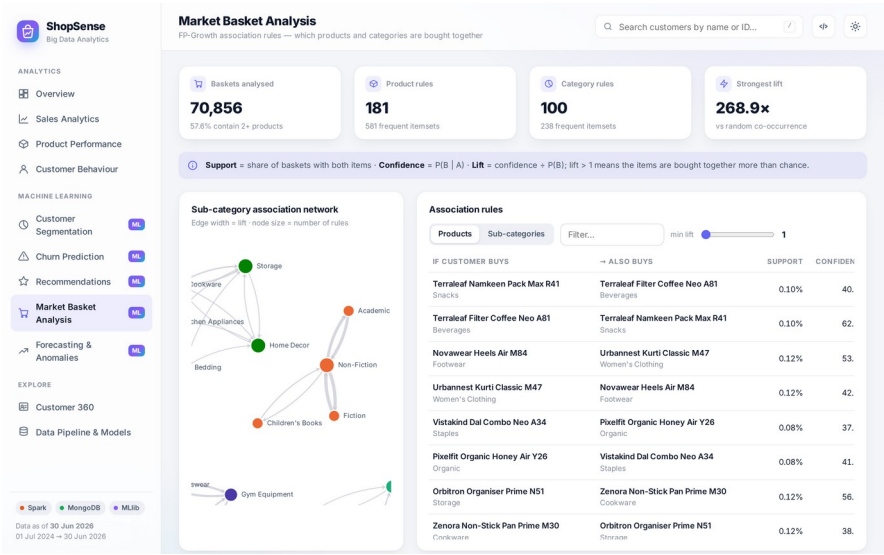


Figure 6.11: Market-basket analysis page

### 6.2.10 Forecasting and anomaly detection

Figure 6.12 shows the forecasting page. For the selected series the chart displays the history, the in-sample model fit and the 60-day forecast with 80% and 95% prediction intervals. Tiles summarise the forecast for the next 30 days and the hold-out accuracy; further cards compare the model with the baselines, check predictions on the hold-out window, list the forecasts of every category, plot the anomaly timeline, validate the detector against the planted events and list unusually large orders and customers on the return-abuse watch-list.

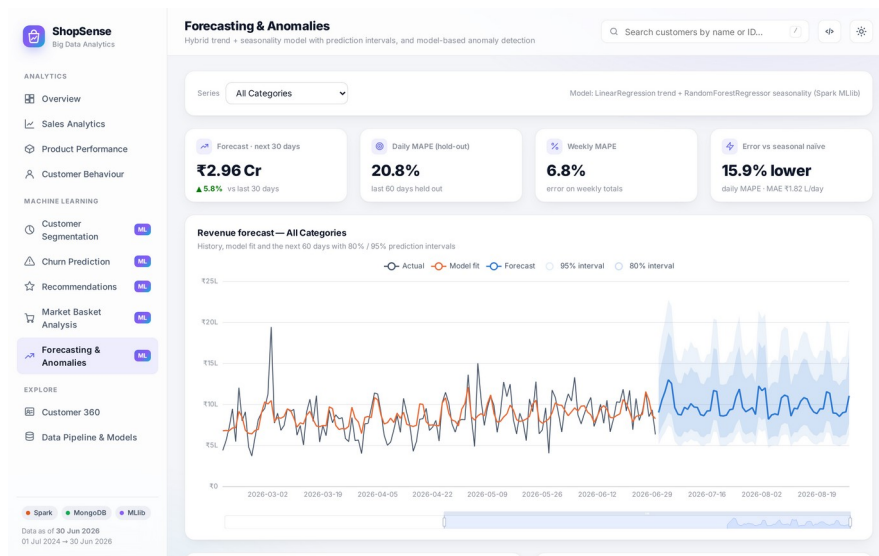


Figure 6.12: Forecasting and anomaly-detection page

### 6.2.11 Customer 360

The Customer 360 page (Figure 6.13) unifies everything known about a customer: profile, lifetime spend, average order value, recency, predicted 12-month value, sessions, returns and cancellations, RFM scores with the segment's recommended action, the churn gauge with the explanation of its drivers, monthly spend, category mix, recent orders and recommendations. Clicking an order opens its embedded line items, read from a single MongoDB document.

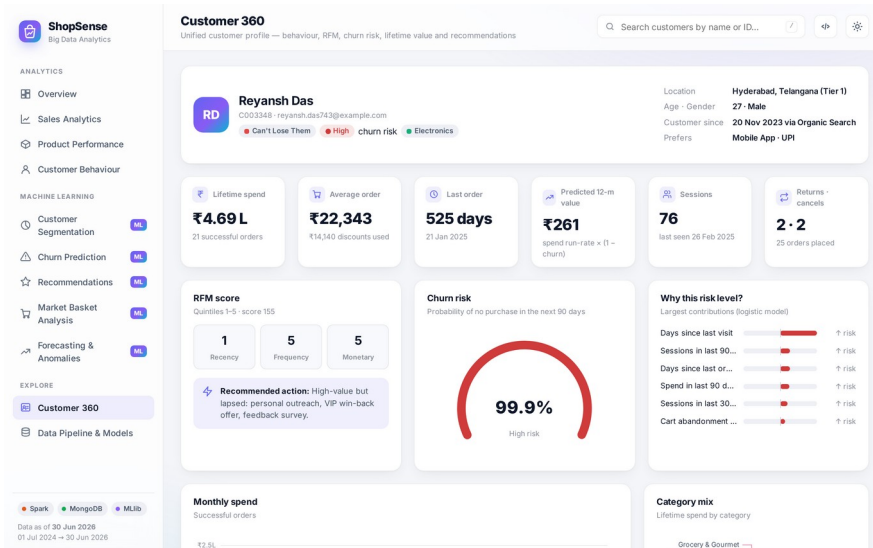


Figure 6.13: Customer 360 page of an at-risk high-value customer

## 6.2.12 Pipeline monitoring and REST API

The pipeline page (Figure 6.14) displays the architecture, the timing of every Spark stage, the data-quality report, the model registry, the MongoDB collection sizes and the run history. All data used by the dashboard is available through the REST API, whose interactive Swagger documentation is generated automatically by FastAPI (Figure 6.15).

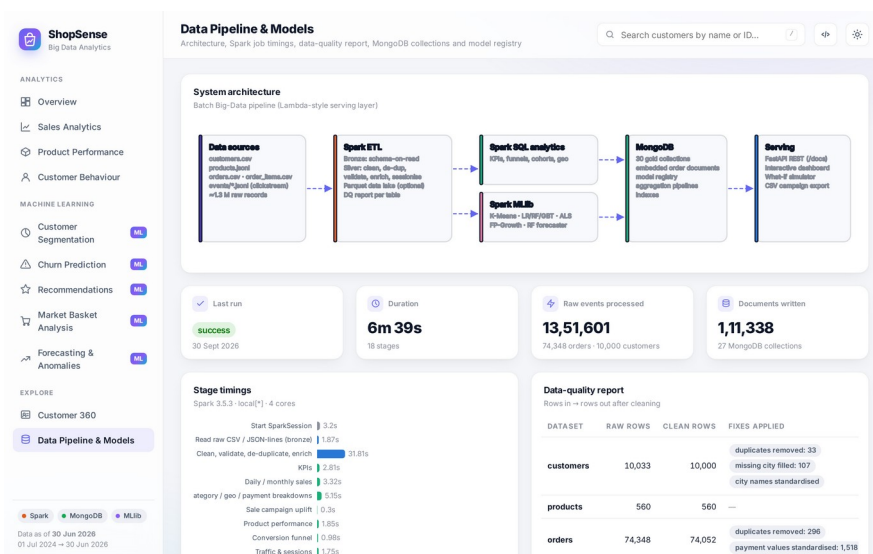


Figure 6.14: Data pipeline and model monitoring page

**Figure 6.15:** Interactive REST API documentation (Swagger UI)

## 6.3 Project Structure

**Table 6.1:** Project modules

| Path                                               | Responsibility                                                                                      |
|----------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| run_pipeline.py,<br>run_dashboard.py               | Command-line entry points of the pipeline (--generate, --scale, --skip-ml) and of the web server    |
| shopsense/config.py,<br>spark_session.py, mongo.py | Configuration (.env), tuned local SparkSession, BSON conversion and distributed MongoDB writer      |
| shopsense/datagen/                                 | Synthetic data generator and reference catalogue (cities, categories, sale calendar)                |
| shopsense/etl/                                     | Bronze ingestion with schemas; silver cleaning, enrichment, sessionisation and data-quality report  |
| shopsense/analytics/                               | Sales KPIs and breakdowns, campaigns, product statistics, funnels, cohorts, orders and Customer 360 |
| shopsense/ml/                                      | Segmentation, churn, recommender, market basket, forecasting and anomaly detection                  |
| shopsense/pipeline.py                              | Stage orchestration, timing and run log                                                             |
| app/main.py, app/static/                           | REST API (29 endpoints) and dashboard (HTML, CSS, JavaScript, bundled ECharts and fonts)            |
| tests/, scripts/, docs/                            | Unit tests; setup, run and capture scripts; documentation, diagrams and screenshots                 |

## 6.4 Key Implementation Details

### 6.4.1 Spark session configuration

The Spark session is tuned for single-machine execution: all cores are used, the number of shuffle partitions is reduced to eight (the default of 200 creates many tiny tasks for

data of this size), adaptive query execution is enabled and the Kryo serializer is used. The Python interpreter of the virtual environment is passed to the workers, which avoids a common problem on Windows.

```
spark =
(SparkSession.builder.appName(app_name).master(config.SPARK_MASTER)
 .config("spark.driver.memory", config.SPARK_DRIVER_MEMORY)
 .config("spark.sql.shuffle.partitions", "8")
 .config("spark.sql.session.timeZone", "Asia/Kolkata")
 .config("spark.sql.adaptive.enabled", "true")
 .config("spark.serializer",
"org.apache.spark.serializer.KryoSerializer")
 .getOrCreate())
```

#### 6.4.2 Distributed write to MongoDB

```
def _write_partition(rows):                                # runs inside every Spark
executor task
    client = MongoClient(uri); target = client[dbname][collection]; batch
= []
    for r in rows:
        batch.append(to_doc(r, id_col))
        if len(batch) >= 2000:
            target.insert_many(batch, ordered=False); batch = []
    if batch: target.insert_many(batch, ordered=False)
    client.close()

df.foreachPartition(_write_partition)
```

#### 6.4.3 Churn model training with cross-validation

```
lr_pipe = Pipeline(stages=_prep_stages() + [
    StandardScaler(inputCol="raw_features", outputCol="features",
withMean=True, withStd=True), lr])
grid = ParamGridBuilder().addGrid(lr.regParam, [0.001, 0.01,
0.1]).build()
cv = CrossValidator(estimator=lr_pipe, estimatorParamMaps=grid,
numFolds=3, seed=42,

evaluator=BinaryClassificationEvaluator(metricName="areaUnderROC"))
fitted["Logistic Regression"] = cv.fit(train).bestModel
fitted["Random Forest"] = Pipeline(stages=_prep_stages() +
[RandomForestClassifier(
    featuresCol="raw_features", numTrees=150, maxDepth=8,
seed=42)]).fit(train)
```

### 6.5 REST API

The FastAPI application exposes 29 endpoints grouped into System, Overview, Sales, Products, Customer Behaviour, Segmentation, Churn, Recommendations, Market Basket, Forecasting, Anomalies and Customer 360. Filters such as date range, category and granularity are evaluated by live MongoDB aggregation pipelines (\$match, \$group, \$dateTrunc, \$unwind) over the gold collections. Examples are *GET /api/sales/trend?*

*start=2026-01-01&category=Fashion&granularity=week*, *POST /api/churn/simulate* with a JSON body of feature values, and *GET /api/churn/customers.csv?risk=High*, which streams a campaign file. The complete reference with request and response examples is provided in the project documentation (docs/API.md) and interactively through Swagger UI.

## 6.6 Testing

Automated unit tests (pytest) cover the data generator, BSON conversion, calendar features and forecast metrics; they run in about three seconds without Spark or MongoDB. Functional testing was carried out by loading every dashboard page in a headless Chromium browser and checking for JavaScript errors, and by calling every API endpoint. Table 6.2 lists representative test cases.

**Table 6.2:** Representative test cases

| ID  | Test case                                             | Expected result                                              | Status |
|-----|-------------------------------------------------------|--------------------------------------------------------------|--------|
| T1  | Generate a 300-customer dataset                       | All files created; duplicate order rows present (dirty data) | Pass   |
| T2  | Convert dates, decimals, NaN and numpy values to BSON | ISO strings, floats, null, native types                      | Pass   |
| T3  | Calendar features for a festive-sale day              | sale_flag = 1 with correct day index                         | Pass   |
| T4  | MAPE / weekly MAPE of a 10% over-forecast             | Both equal 10%                                               | Pass   |
| T5  | Run pipeline with MongoDB stopped                     | Clear error message, no crash                                | Pass   |
| T6  | Run full pipeline (medium scale)                      | All 18 stages succeed; run logged                            | Pass   |
| T7  | Data-quality: count duplicates removed                | Raw – clean equals injected duplicates                       | Pass   |
| T8  | All 29 API endpoints                                  | HTTP 200 with valid JSON / CSV                               | Pass   |
| T9  | Load all 11 dashboard pages (light and dark)          | No JavaScript errors; charts rendered                        | Pass   |
| T10 | What-if simulator: increase days since last visit     | Churn probability increases                                  | Pass   |
| T11 | Search customer by partial name                       | Matching customers listed; Customer 360 opens                | Pass   |
| T12 | Export high-risk customers to CSV                     | File downloaded with 12 columns                              | Pass   |
| T13 | Unknown customer ID                                   | HTTP 404                                                     | Pass   |

## CHAPTER 7

### RESULT ANALYSIS

#### 7.1 Introduction

This chapter evaluates ShopSense quantitatively: the performance and data quality of the pipeline, the business insights produced by the descriptive analytics and the accuracy of each machine-learning model against appropriate baselines. All results were obtained on the default medium dataset (10,000 customers, 74,348 orders, 13,51,601 events) with Spark 3.5.3 in local mode on 4 CPU cores, using fixed random seeds.

#### 7.2 Performance Measures

##### 7.2.1 Pipeline performance and scalability

Table 7.1 lists the duration of every pipeline stage. Ingestion and cleaning of about 1.4 million raw records takes about half a minute, all descriptive analytics together take about 25 seconds and machine learning accounts for most of the run time, dominated by the cross-validated churn models and the ALS grid search.

**Table 7.1:** Pipeline stage timings (medium dataset)

| Group     | Stage                                 | Time (s) | Documents written |
|-----------|---------------------------------------|----------|-------------------|
| Setup     | Start SparkSession                    | 3.2      | –                 |
| ETL       | Read raw CSV / JSON-lines (bronze)    | 1.9      | –                 |
| ETL       | Clean, validate, de-duplicate, enrich | 31.8     | –                 |
| Analytics | KPIs                                  | 2.8      | 1                 |
| Analytics | Daily / monthly sales                 | 3.3      | 6,594             |
| Analytics | Category / geo / payment breakdowns   | 5.2      | 284               |
| Analytics | Sale campaign uplift                  | 0.3      | 18                |
| Analytics | Product performance                   | 1.9      | 560               |
| Analytics | Conversion funnel                     | 1.0      | 10                |
| Analytics | Traffic & sessions                    | 1.8      | 55                |
| Analytics | Cohort retention                      | 1.7      | 24                |
| Analytics | Orders collection (embedded items)    | 3.8      | 74,009            |
| Analytics | Customer 360 profiles                 | 2.6      | 10,000            |
| ML        | Segmentation - K-Means + RFM          | 33.3     | 6,037             |
| ML        | Churn - LR / RF / GBT                 | 152.7    | 6,037             |

| Group | Stage                        | Time (s) | Documents written |
|-------|------------------------------|----------|-------------------|
| ML    | Recommendations - ALS        | 96.9     | 7,419             |
| ML    | Market basket - FP-Growth    | 3.5      | 281               |
| ML    | Forecast + anomaly detection | 51.6     | 9                 |
|       | Total                        | 399.1    |                   |

To assess scalability, the pipeline was run on all three dataset sizes (Table 7.2). Increasing the data ten-fold from *small* to *large* increases the run time far less than proportionally, because much of the time is fixed Spark job-scheduling overhead of the iterative ML algorithms, while the data-parallel stages scale almost linearly. The same code can therefore handle considerably larger data on a cluster.

**Table 7.2:** Scalability across dataset sizes

| Scale  | Customers | Orders   | Events    | Raw size | Pipeline time |
|--------|-----------|----------|-----------|----------|---------------|
| Small  | 2,500     | 18,881   | 3,41,198  | 80 MB    | 5.3 min       |
| Medium | 10,000    | 74,052   | 13,48,916 | 315 MB   | 6.7 min       |
| Large  | 25,000    | 1,86,698 | 33,88,035 | 790 MB   | 9.1 min       |

### 7.2.2 Data quality

Table 7.3 shows the effect of the cleaning stage. Every injected problem was detected and corrected: duplicate customers, orders and events were removed, 1,518 inconsistent payment-method values and 3,362 event types with wrong casing were standardised, 107 missing cities were filled and 208 line items with invalid quantities were discarded.

**Table 7.3:** Data-quality report

| Dataset            | Raw rows  | Clean rows | Corrections applied                                                       |
|--------------------|-----------|------------|---------------------------------------------------------------------------|
| Customers          | 10,033    | 10,000     | 33 duplicates removed; 107 missing cities filled; city names standardised |
| Products           | 560       | 560        | Validated (no invalid prices)                                             |
| Orders             | 74,348    | 74,052     | 296 duplicates removed; 1,518 payment values standardised                 |
| Order items        | 1,45,368  | 1,45,160   | 208 invalid quantities removed                                            |
| Events             | 13,51,601 | 13,47,800  | 2,685 duplicates; 3,362 casing fixes; 1,116 invalid events removed        |
| Sessions (derived) | –         | 2,86,848   | Sessionised from clean events                                             |

### 7.2.3 Business analytics results

Table 7.4 summarises the headline KPIs computed by the gold layer.

**Table 7.4:** Key performance indicators (July 2024 – June 2026)

| KPI                 | Value     | KPI                     | Value            |
|---------------------|-----------|-------------------------|------------------|
| Net revenue         | ₹45.94 Cr | Gross profit (margin)   | ₹7.58 Cr (16.5%) |
| Successful orders   | 66,217    | Orders placed           | 74,009           |
| Average order value | ₹6,938    | Units sold              | 1,47,335         |
| Buying customers    | 6,037     | Repeat-customer rate    | 83.3%            |
| Sessions            | 2,86,848  | Session conversion rate | 25.8%            |
| Cart abandonment    | 34.6%     | Bounce rate             | 29.0%            |
| Cancellation rate   | 4.3%      | Return rate             | 6.3%             |
| Average rating      | 3.98 / 5  | Discounts given         | ₹3.67 Cr         |

Category analysis (Table 7.5) reveals a classic trade-off: Electronics generates 60.4% of revenue but earns only 7.3% margin, while Fashion and Beauty & Personal Care earn margins of about 36–40% but Fashion also has the highest return rate (11.9%). Promoting high-margin categories to Electronics buyers is therefore an obvious cross-selling opportunity.

**Table 7.5:** Category performance

| Category               | Revenue   | Share | Margin | Avg. discount | Return rate | Rating |
|------------------------|-----------|-------|--------|---------------|-------------|--------|
| Electronics            | ₹27.76 Cr | 60.4% | 7.3%   | 7.7%          | 5.8%        | 4.08   |
| Fashion                | ₹5.94 Cr  | 12.9% | 36.1%  | 7.5%          | 11.9%       | 4.07   |
| Sports & Fitness       | ₹3.98 Cr  | 8.7%  | 26.2%  | 7.6%          | 6.9%        | 3.83   |
| Beauty & Personal Care | ₹3.35 Cr  | 7.3%  | 40.1%  | 7.5%          | 5.2%        | 3.95   |
| Home & Kitchen         | ₹2.90 Cr  | 6.3%  | 24.6%  | 7.7%          | 6.8%        | 3.89   |
| Toys & Baby            | ₹83.14 L  | 1.8%  | 28.1%  | 7.8%          | 6.4%        | 3.89   |
| Grocery & Gourmet      | ₹81.41 L  | 1.8%  | 4.5%   | 7.6%          | 4.5%        | 4.07   |
| Books                  | ₹36.39 L  | 0.8%  | 14.9%  | 8.0%          | 4.9%        | 3.91   |

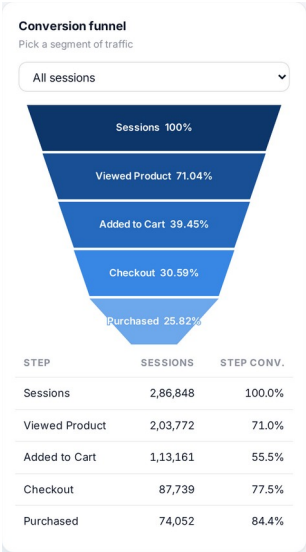
Payment analysis shows that UPI is the dominant payment method, while **Cash on Delivery orders are cancelled 7.4% of the time**, roughly twice the rate of prepaid methods (about 3–4%). Figure 7.1 shows the uplift of every sale event: order volume

risers in every event (up to +117% during the Big Festive Sale), but deep discounts reduce the *revenue* uplift considerably and sometimes make it negative, an insight that only becomes visible when both measures are compared.



**Figure 7.1:** Order-volume uplift versus revenue uplift of sale events

The conversion funnel (Figure 7.2) shows that 71.0% of sessions view a product, 39.5% add to cart, 30.6% reach checkout and 25.8% purchase; the largest drop occurs between product view and add-to-cart, which is where merchandising and recommendation efforts should concentrate.



**Figure 7.2:** Conversion funnel for all sessions

### 7.2.4 Customer segmentation

Table 7.6 lists the silhouette coefficient for  $k = 3 \dots 8$ . The best value is obtained at  $k = 5$  ( $S = 0.3746$ ), which also falls in the interpretable range. The resulting segments are described in Table 7.7.

**Table 7.6:** Silhouette coefficient for different numbers of clusters

| k          | 3      | 4      | 5      | 6      | 7      | 8      |
|------------|--------|--------|--------|--------|--------|--------|
| Silhouette | 0.3718 | 0.3538 | 0.3746 | 0.3549 | 0.3624 | 0.3526 |
| WSSSE      | 24,026 | 20,693 | 18,045 | 16,385 | 15,095 | 14,068 |

**Table 7.7:** Customer segments discovered by K-Means

| Segment                 | Customers | Avg. recency (days) | Avg. orders | Avg. spend | Revenue share |
|-------------------------|-----------|---------------------|-------------|------------|---------------|
| Champions               | 1,394     | 45                  | 30.1        | ₹2.36 L    | 71.6%         |
| Can't Lose Them         | 1,541     | 360                 | 7.9         | ₹45,339    | 15.2%         |
| Loyal Customers         | 1,406     | 38                  | 6.4         | ₹39,102    | 12.0%         |
| Lost                    | 1,223     | 300                 | 1.7         | ₹3,178     | 0.8%          |
| Hibernating             | 473       | 344                 | 1.9         | ₹3,863     | 0.4%          |
| Prospects (no purchase) | 3,963     | –                   | 0           | –          | 0%            |

The segmentation exposes a strong Pareto effect: *Champions* form 23.1% of buyers but generate 71.6% of revenue, while the *Can't Lose Them* segment (high historical value, last order about a year ago) holds 15.2% of revenue and is the natural target of a VIP win-back campaign.

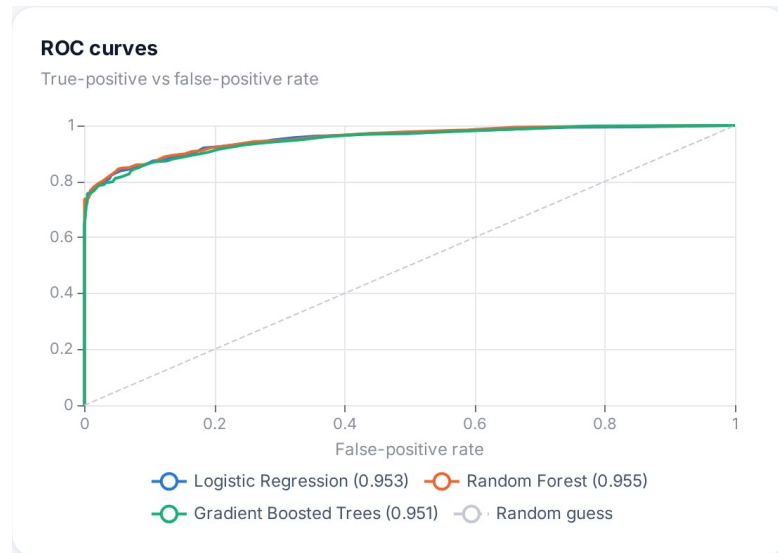
### 7.2.5 Churn prediction

Table 7.8 compares the three classifiers on the held-out test set. All three achieve an ROC-AUC above 0.95, which shows that the engineered behavioural features carry a strong signal. **Random Forest** obtains the best AUC (0.955), PR-AUC (0.972), accuracy (88.5%) and F1-score (0.8956), with a precision of 93.5%: more than nine out of ten customers flagged as churners really churned. Logistic regression is only marginally worse and offers the best recall, which justifies using it for explanations.

**Table 7.8:** Churn model comparison on the test set

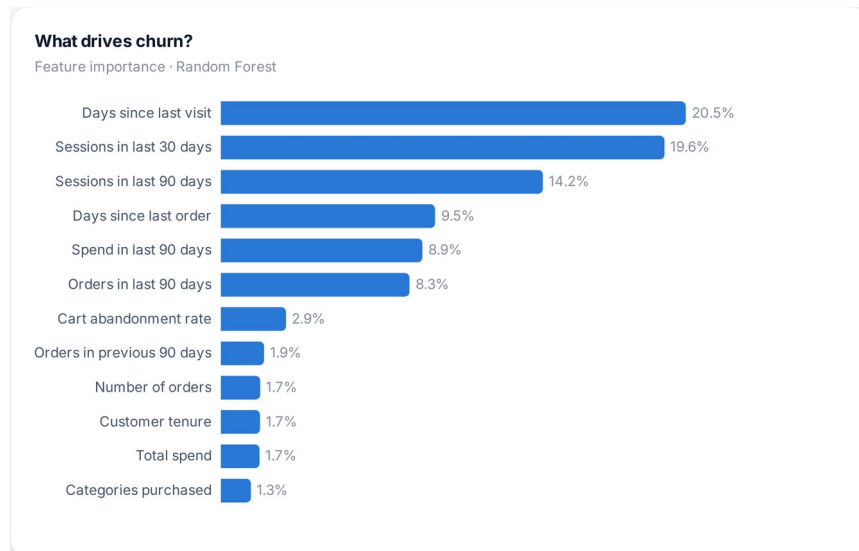
| Model                | ROC-AUC | PR-AUC | Accuracy | Precision | Recall | F1    |
|----------------------|---------|--------|----------|-----------|--------|-------|
| Logistic Regression  | 0.953   | 0.971  | 88.0%    | 91.0%     | 87.7%  | 0.893 |
| Random Forest (best) | 0.955   | 0.972  | 88.5%    | 93.5%     | 85.9%  | 0.896 |

| Model                  | ROC-AUC | PR-AUC | Accuracy | Precision | Recall | F1    |
|------------------------|---------|--------|----------|-----------|--------|-------|
| Gradient Boosted Trees | 0.951   | 0.969  | 87.8%    | 93.3%     | 84.8%  | 0.888 |



**Figure 7.3:** ROC curves of the three churn models

Figure 7.4 shows the most important features. The top three are *days since last visit* (20.5%), *sessions in the last 30 days* (19.6%) and *sessions in the last 90 days* (14.2%): **click-stream engagement predicts churn better than purchase history alone**, which underlines the value of processing behavioural log data with Big Data tools. On the test set the best model produced 536 true positives, 428 true negatives, 37 false positives and 88 false negatives, a balanced error profile. Scoring all customers as of the reference date places 2,956 customers in the high-risk band, representing ₹8.37 Cr of annual revenue at risk.



**Figure 7.4:** Feature importance of the Random Forest churn model

### 7.2.6 Product recommendations

Table 7.9 reports the leave-last-purchase-out evaluation on 4,383 customers. The tuned ALS model (rank 64,  $\lambda = 1$ ,  $\alpha = 5$ ) places the hidden next purchase in its Top-10 list for **18.0%** of customers, **1.8×** the hit rate of the popularity baseline, and nearly doubles NDCG@10. The grid search shows that strong regularisation with a moderate confidence weight is essential on sparse implicit data: the default-style configuration (rank 16,  $\lambda = 0.05$ ,  $\alpha = 15$ ) barely beats popularity.

**Table 7.9:** Recommendation accuracy (Top-10, leave-last-purchase-out)

| Model               | Rank | $\lambda$ | $\alpha$ | HitRate@10 | NDCG@10 |
|---------------------|------|-----------|----------|------------|---------|
| Popularity baseline | —    | —         | —        | 9.99%      | 5.11%   |
| ALS                 | 16   | 0.05      | 15       | 11.36%     | 5.25%   |
| ALS                 | 32   | 1         | 5        | 16.93%     | 8.74%   |
| ALS (chosen)        | 64   | 1         | 5        | 18.00%     | 9.33%   |
| ALS                 | 64   | 1         | 10       | 17.82%     | 9.13%   |

### 7.2.7 Market-basket analysis

FP-Growth analysed 70,856 baskets, 57.6% of which contain two or more products, and produced 181 product-level and 100 sub-category-level rules with lift greater than one. Table 7.10 lists the strongest sub-category rules; they correspond to the complementary relationships built into the generator (for example cookware with kitchen appliances),

which validates the method. Product-level rules reach confidences above 80% and are used for the “frequently bought together” widgets.

**Table 7.10:** Strongest sub-category association rules

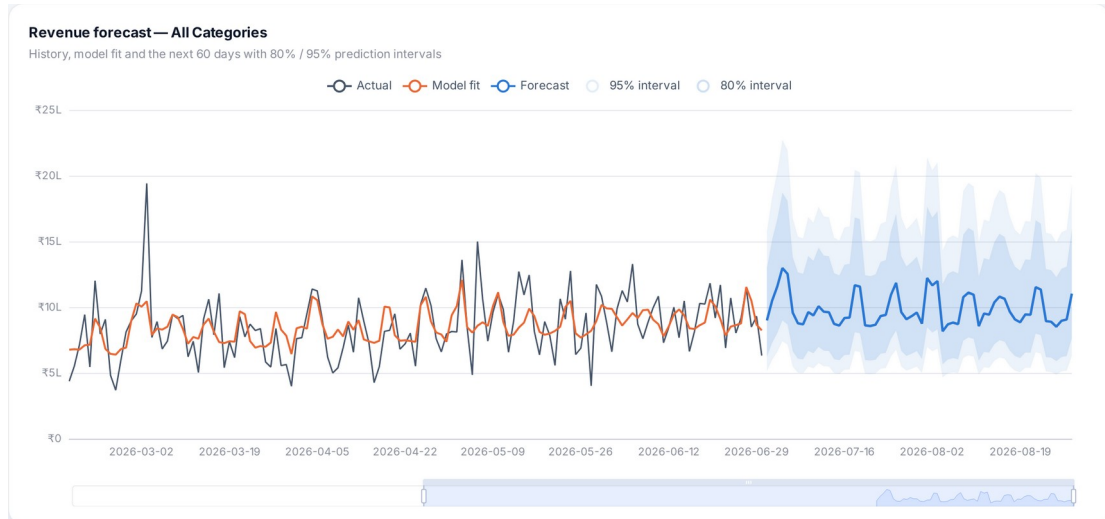
| Antecedent         | Consequent         | Support | Confidence | Lift  |
|--------------------|--------------------|---------|------------|-------|
| Cookware           | Kitchen Appliances | 2.09%   | 40.8%      | 10.51 |
| Kitchen Appliances | Cookware           | 2.09%   | 53.8%      | 10.51 |
| Non-Fiction        | Fiction            | 1.09%   | 25.8%      | 8.55  |
| Fiction            | Non-Fiction        | 1.09%   | 36.2%      | 8.55  |
| Toys               | Baby Care          | 1.35%   | 29.7%      | 7.95  |
| Baby Care          | Toys               | 1.35%   | 36.2%      | 7.95  |
| Academic           | Non-Fiction        | 0.68%   | 31.1%      | 7.33  |
| Non-Fiction        | Academic           | 0.68%   | 16.1%      | 7.33  |

### 7.2.8 Sales forecasting

Table 7.11 compares the hybrid forecaster with three baselines on the 60-day hold-out window. The hybrid model has the lowest daily MAPE (20.8%), the lowest MAE and the highest  $R^2$ . Day-level  $R^2$  is close to zero for all models because daily revenue is dominated by a small number of very expensive baskets (a single laptop order can equal a hundred grocery orders), which no model can predict; aggregated to weekly totals the hybrid model’s error falls to 6.8%. Importantly, only the hybrid model anticipates upcoming sale events because it uses the marketing calendar. The forecast for the next 30 days is ₹2.96 Cr compared with ₹2.80 Cr in the last 30 days.

**Table 7.11:** Forecast accuracy on the 60-day hold-out window

| Model                            | Daily MAPE | Weekly MAPE | MAE (₹/day) | $R^2$  |
|----------------------------------|------------|-------------|-------------|--------|
| Hybrid LR-trend + RF-seasonality | 20.8%      | 6.8%        | 1,81,955    | -0.060 |
| Calendar Linear Regression       | 22.7%      | 7.6%        | 1,95,550    | -0.134 |
| Seasonal Naive (last week)       | 24.7%      | 6.3%        | 2,18,681    | -0.448 |
| 28-day Moving Average            | 21.9%      | 14.4%       | 2,08,054    | -0.367 |



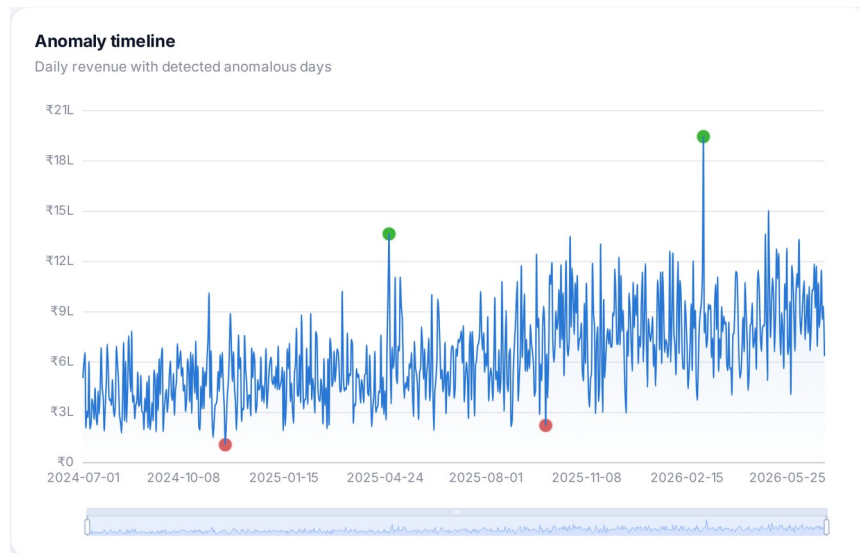
**Figure 7.5:** Revenue forecast with 80% and 95% prediction intervals

### 7.2.9 Anomaly detection

The anomaly detector flagged exactly 4 days and recovered **all 4 planted anomalies (recall 100%) with no false alarms** (Table 7.12), even though the series contains nine kinds of sale events with order volumes up to twice the normal level. A naïve detector based on the mean and standard deviation of daily revenue would flag every festival and miss the drops. The order-level checks additionally flagged 40 unusually large orders and 5 customers with suspicious return behaviour.

**Table 7.12:** Detected anomalous days versus planted ground truth

| Date       | Planted event                 | Orders (actual / expected) | Robust z | Detected |
|------------|-------------------------------|----------------------------|----------|----------|
| 2024-11-18 | Payment gateway outage        | 20 / 50.1                  | -7.16    | Yes      |
| 2025-04-28 | Viral influencer flash sale   | 217 / 84.5                 | 7.78     | Yes      |
| 2025-09-29 | Website downtime              | 33 / 76.6                  | -6.64    | Yes      |
| 2026-03-03 | Bulk corporate gifting orders | 293 / 132.1                | 6.60     | Yes      |



**Figure 7.6:** Anomaly timeline: daily revenue with detected anomalies

### 7.3 Screenshots

Screenshots of every module of the running system, together with a description of each screen, are presented in Section 6.2 (Figures 6.1 – 6.15), and the charts supporting the performance measures are included in Section 7.2. All screens are also available in dark and light themes in the project repository (docs/screenshots) together with an animated tour and a demo video.

### 7.4 Comparative Analysis

Table 7.13 compares ShopSense with the approaches reviewed in Chapter 2 and with the tools commonly used by small retailers. The comparison is qualitative for published studies because they use different datasets; quantitative comparisons were made against standard baselines on the same data (popularity recommender, seasonal-naïve and moving-average forecasts, mean/standard-deviation thresholds).

**Table 7.13:** Comparison with existing approaches

| Aspect            | Existing approaches                                         | ShopSense                                                                                        |
|-------------------|-------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| Data processing   | Spreadsheets or single-server SQL; MapReduce batch jobs [3] | In-memory Spark DataFrames and Spark SQL [1][2][4]; 1.4 M records in $\approx 7$ min on a laptop |
| Data quality      | Usually assumes pre-cleaned data                            | Measured cleaning of duplicates, casing, nulls and invalid rows with a DQ report                 |
| Storage / serving | Relational reports regenerated per query                    | Query-shaped MongoDB documents with embedding and indexes; millisecond API responses             |

| Aspect                  | Existing approaches                        | ShopSense                                                                                                        |
|-------------------------|--------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| Segmentation            | RFM + k-means with manual labels [7]       | RFM + k-means with silhouette selection, automatic business naming and action playbook                           |
| Churn prediction        | Random snapshot splits; accuracy only [11] | Time-based leak-free split, 3 models, AUC 0.955, explanations and what-if simulator                              |
| Recommendation          | Explicit ratings [15] or best-seller lists | Implicit-feedback ALS [16] with offline evaluation: 1.8× the popularity baseline                                 |
| Market basket           | Apriori with candidate generation [17]     | Parallel FP-Growth [18][19] at two levels; rules embedded in product pages                                       |
| Forecasting             | Moving averages or trend only              | Trend + calendar seasonality with sale events, smearing correction and intervals; lowest daily error of 4 models |
| Anomaly detection       | Fixed thresholds or mean $\pm 3\sigma$     | Model-residual robust z-score (MAD) [23]; 4/4 planted anomalies, 0 false alarms                                  |
| Integration & usability | Separate tools for each task               | Single pipeline and 11-page dashboard with Customer 360, CSV export and REST API                                 |
| Cost                    | Commercial BI / analytics licences         | Completely open-source; runs on existing hardware                                                                |

## CHAPTER 8

### CONCLUSION & FUTURE SCOPE

#### 8.1 Conclusion

This project designed and implemented **ShopSense**, an end-to-end Big Data analytics platform for e-commerce sales and customer behaviour built with Apache Spark, MongoDB and machine learning. A behaviour-driven data generator produced a realistic two-year dataset of about 1.4 million records containing seasonality, festive sales, customer life-cycles and deliberate data-quality problems. A layered Spark pipeline ingested the CSV and JSON-lines data, cleaned and sessionised it with a measurable data-quality report and computed descriptive analytics, and a partition-parallel loader stored query-ready documents in MongoDB. Six Spark MLlib models then added intelligence: K-Means segmentation revealed that 23% of buyers generate 72% of revenue; a leak-free Random Forest churn model reached an ROC-AUC of 0.955 and identified click-stream engagement as the strongest churn signal; implicit-feedback ALS recommendations were 1.8 times more accurate than a popularity baseline; FP-Growth recovered meaningful bundles; the calendar-aware hybrid forecaster achieved the lowest error among four models; and the robust anomaly detector found every planted anomaly without false alarms.

All results are delivered through a documented REST API and an interactive dashboard that translates analytics into decisions: segment playbooks, explained churn scores, a what-if simulator, exportable win-back lists, personalised recommendations and forecasts with uncertainty. The complete pipeline runs in about seven minutes on a laptop with only open-source software, demonstrating that the Big Data and machine-learning techniques studied in the curriculum can deliver practical, explainable and affordable business value.

#### 8.2 Future Scope

- **Real-time streaming:** ingest click-stream events with Apache Kafka and Spark Structured Streaming to update funnels and anomaly alerts within seconds.

- **Cluster deployment:** run Spark on a multi-node cluster (YARN or Kubernetes), store the lake on HDFS or cloud object storage with Delta Lake, and use a MongoDB replica set or Atlas for high availability.
- **Real datasets:** apply the pipeline to public datasets such as UCI Online Retail II or large e-commerce event logs, or to a partner retailer's anonymised data.
- **Advanced models:** survival analysis for time-to-churn, uplift modelling to target only customers who respond to offers, sequence-aware or graph-neural recommenders, and deep-learning or gradient-boosted forecasters with external regressors.
- **Review and sentiment analytics:** add product reviews and use NLP to relate sentiment to returns and churn.
- **MLOps:** schedule the pipeline with Apache Airflow, track experiments with MLflow, monitor data and model drift and retrain automatically.
- **Security and multi-user access:** authentication, role-based dashboards and audit logs for enterprise deployment.
- **Conversational analytics:** allow managers to query the data in natural language through a large-language-model assistant grounded in the MongoDB collections.

## REFERENCES

- [1] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauley, M. J. Franklin, S. Shenker and I. Stoica, “Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing,” in Proc. 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI), San Jose, CA, 2012, pp. 15–28.
- [2] M. Zaharia, R. S. Xin, P. Wendell, T. Das, M. Armbrust, A. Dave, X. Meng, J. Rosen, S. Venkataraman, M. J. Franklin, A. Ghodsi, J. Gonzalez, S. Shenker and I. Stoica, “Apache Spark: A Unified Engine for Big Data Processing,” *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016.
- [3] J. Dean and S. Ghemawat, “MapReduce: Simplified Data Processing on Large Clusters,” in Proc. 6th USENIX Symposium on Operating Systems Design and Implementation (OSDI), San Francisco, CA, 2004, pp. 137–150.
- [4] M. Armbrust, R. S. Xin, C. Lian, Y. Huai, D. Liu, J. K. Bradley, X. Meng, T. Kaftan, M. J. Franklin, A. Ghodsi and M. Zaharia, “Spark SQL: Relational Data Processing in Spark,” in Proc. ACM SIGMOD International Conference on Management of Data, Melbourne, 2015, pp. 1383–1394.
- [5] X. Meng, J. Bradley, B. Yavuz, E. Sparks, S. Venkataraman, D. Liu, J. Freeman, D. B. Tsai, M. Amde, S. Owen, D. Xin, R. Xin, M. J. Franklin, R. Zadeh, M. Zaharia and A. Talwalkar, “MLlib: Machine Learning in Apache Spark,” *Journal of Machine Learning Research*, vol. 17, no. 34, pp. 1–7, 2016.
- [6] S. Bradshaw, E. Brazil and K. Chodorow, *MongoDB: The Definitive Guide*, 3rd ed. Sebastopol, CA, USA: O’Reilly Media, 2019.
- [7] D. Chen, S. L. Sain and K. Guo, “Data mining for the online retail industry: A case study of RFM model-based customer segmentation using data mining,” *Journal of Database Marketing & Customer Strategy Management*, vol. 19, no. 3, pp. 197–208, 2012.
- [8] P. S. Fader, B. G. S. Hardie and K. L. Lee, “RFM and CLV: Using Iso-Value Curves for Customer Base Analysis,” *Journal of Marketing Research*, vol. 42, no. 4, pp. 415–430, 2005.

- [9] B. Bahmani, B. Moseley, A. Vattani, R. Kumar and S. Vassilvitskii, “Scalable K-Means++,” *Proceedings of the VLDB Endowment*, vol. 5, no. 7, pp. 622–633, 2012.
- [10] P. J. Rousseeuw, “Silhouettes: A graphical aid to the interpretation and validation of cluster analysis,” *Journal of Computational and Applied Mathematics*, vol. 20, pp. 53–65, 1987.
- [11] W. Verbeke, K. Dejaeger, D. Martens, J. Hur and B. Baesens, “New insights into churn prediction in the telecommunication sector: A profit driven data mining approach,” *European Journal of Operational Research*, vol. 218, no. 1, pp. 211–229, 2012.
- [12] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [13] J. H. Friedman, “Greedy Function Approximation: A Gradient Boosting Machine,” *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.
- [14] T. Fawcett, “An introduction to ROC analysis,” *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.
- [15] Y. Koren, R. Bell and C. Volinsky, “Matrix Factorization Techniques for Recommender Systems,” *Computer*, vol. 42, no. 8, pp. 30–37, 2009.
- [16] Y. Hu, Y. Koren and C. Volinsky, “Collaborative Filtering for Implicit Feedback Datasets,” in *Proc. 8th IEEE International Conference on Data Mining (ICDM)*, Pisa, 2008, pp. 263–272.
- [17] R. Agrawal and R. Srikant, “Fast Algorithms for Mining Association Rules,” in *Proc. 20th International Conference on Very Large Data Bases (VLDB)*, Santiago, 1994, pp. 487–499.
- [18] J. Han, J. Pei and Y. Yin, “Mining Frequent Patterns without Candidate Generation,” in *Proc. ACM SIGMOD International Conference on Management of Data*, Dallas, TX, 2000, pp. 1–12.
- [19] H. Li, Y. Wang, D. Zhang, M. Zhang and E. Y. Chang, “PFP: Parallel FP-Growth for Query Recommendation,” in *Proc. ACM Conference on Recommender Systems (RecSys)*, Lausanne, 2008, pp. 107–114.
- [20] S. J. Taylor and B. Letham, “Forecasting at Scale,” *The American Statistician*, vol. 72, no. 1, pp. 37–45, 2018.

- [21] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021. [Online]. Available: <https://otexts.com/fpp3/>
- [22] V. Chandola, A. Banerjee and V. Kumar, “Anomaly Detection: A Survey,” *ACM Computing Surveys*, vol. 41, no. 3, Art. 15, 2009.
- [23] C. Leys, C. Ley, O. Klein, P. Bernard and L. Licata, “Detecting outliers: Do not use standard deviation around the mean, use absolute deviation around the median,” *Journal of Experimental Social Psychology*, vol. 49, no. 4, pp. 764–766, 2013.
- [24] N. Duan, “Smearing Estimate: A Nonparametric Retransformation Method,” *Journal of the American Statistical Association*, vol. 78, no. 383, pp. 605–610, 1983.
- [25] M. Armbrust, A. Ghodsi, R. Xin and M. Zaharia, “Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics,” in *Proc. 11th Conference on Innovative Data Systems Research (CIDR)*, 2021.
- [26] B. W. Boehm, *Software Engineering Economics*. Englewood Cliffs, NJ, USA: Prentice-Hall, 1981.
- [27] The Apache Software Foundation, “Apache Spark 3.5.3 Documentation: Machine Learning Library (MLlib) Guide.” [Online]. Available: <https://spark.apache.org/docs/3.5.3/ml-guide.html>
- [28] MongoDB Inc., “MongoDB Manual: Aggregation Operations.” [Online]. Available: <https://www.mongodb.com/docs/manual/aggregation/>
- [29] S. Ramírez, “FastAPI Documentation.” [Online]. Available: <https://fastapi.tiangolo.com/>
- [30] Apache ECharts, “Apache ECharts Documentation.” [Online]. Available: <https://echarts.apache.org/>

## **PUBLISHED PAPERS**

*(Details of any paper published or communicated based on this project, together with a copy of the paper, are to be attached here.)*

## **CERTIFICATES**

*(Paper publication certificates / conferences attended / competitions participated, if any, are to be attached here.)*

## **PLAGIARISM REPORT**

*(The first few pages of the plagiarism report generated by the institute-approved tool  
are to be attached here.)*